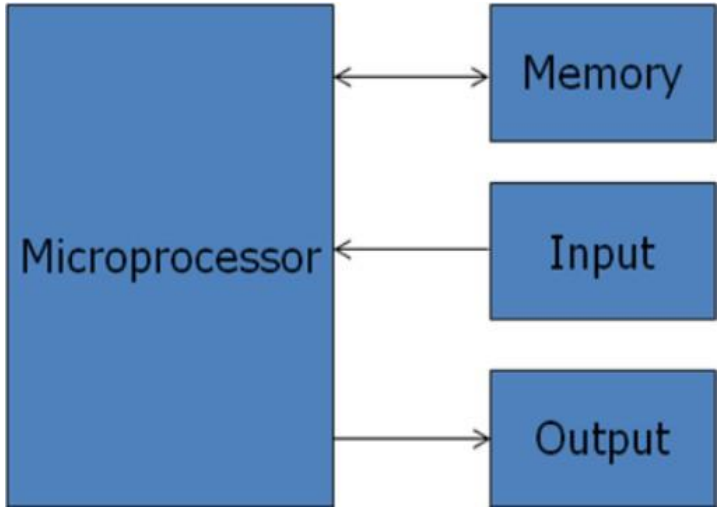
UNIT-I

Introduction to Microprocessor (9 Hours)

Contents

Microprocessor

- Multipurpose, programmable, clock driven, register based integrated circuit fabricated using LSI, VLSI etc. technology.
- Has limited set of onchip memory location called registers to hold information.
- Clock synchronizes all the components present in microprocessor.
- Can understand fixed set of basic commands and can generate signals to control external devices.
- When interfaced with other devices, it can read instructions from memory, accepts data from input devices and process data according to the instructions and provide output.
- Communicates with the peripherals (memory, I/Os) using system bus.



LSI: Large Scale Integration
VLSI: Very Large Scale Integration

Core Components of a Computer System: CPU, MPU & Microcontroller

- **CPU**: - Central processing unit which consists of ALU and control unit.
- **Microprocessor:** - Single chip containing all units of CPU.
- **Microcomputer:** - Computer having microprocessor as CPU.
- **Microcontroller:** single chip consisting of MPU, memory, I/O and interfacing circuits.
- **MPU:** - Microprocessing unit – complete processing unit with the necessary control signals.

Term	Key Idea
CPU	Brain of computer (ALU + CU)
Microprocessor	CPU on a single chip
Microcomputer	Computer using a microprocessor
Microcontroller	Complete system on a chip
MPU	Same as microprocessor

Applications of Microprocessor

- Microcomputers
- Industrial Control
- Robotics
- Traffic Lights
- Washing Machines
- Microwave Oven
- Security Systems
- On Board Systems etc.

Advantages of Microprocessor

- Computational/Processing speed is high
- Intelligence has been brought to systems
- Automation of industrial process and office automation
- Flexible
- Compact in size
- Maintenance is easier

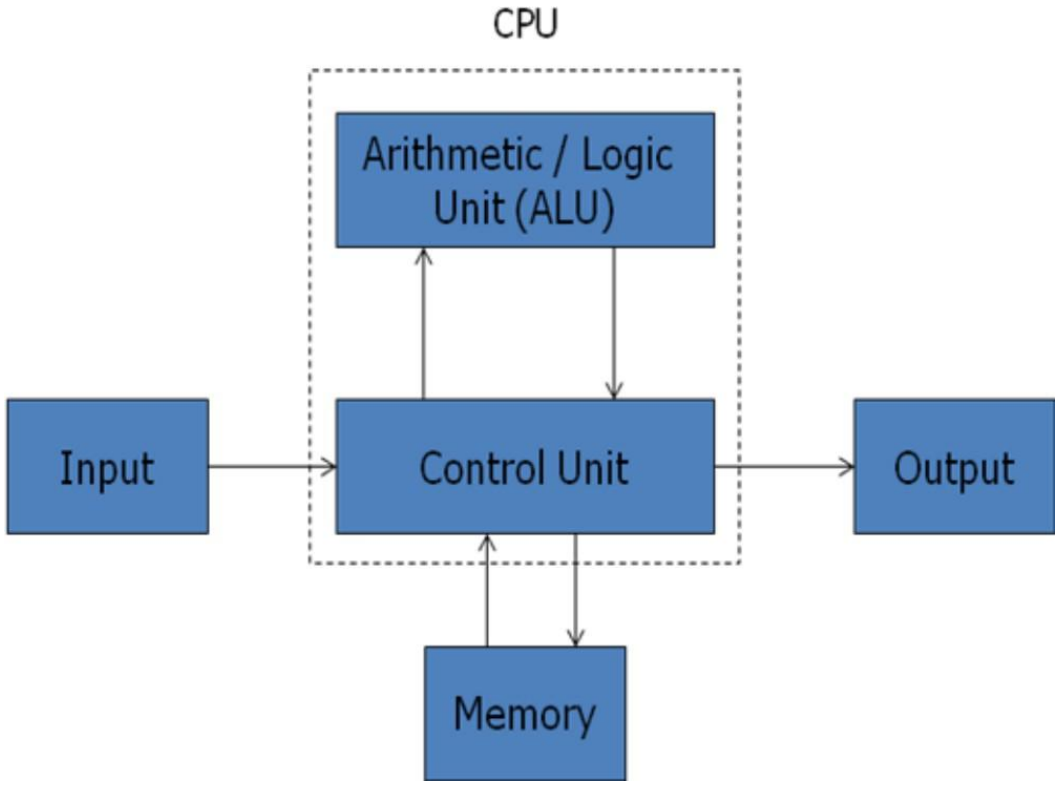
History of Microprocessor

Microprocessor Generations: Intel & AMD					
Generation	**Year**	**Processor Models**	**Data Bus**	**Address Bus**	**Max Memory**
1st (4-bit)	1971	Intel 4004 / 4040	4-bit	12-bit	4 KB
1st (8-bit)	1972	Intel 8008	8-bit	14-bit	16 KB
2nd (8-bit)	1974	Intel 8080 / 8085	8-bit	16-bit	64 KB
3rd (16-bit)	1978	Intel 8086 / 8088 / 80286	16-bit	20 to 24-bit	1 to 16 MB
4th (32-bit)	1985	Intel 80386 / 80486, AMD Am486	32-bit	32-bit	4 GB
5th (Pentium)	1993	Intel Pentium / Pentium Pro, AMD K5	64-bit	32 to 36-bit	4 to 64 GB
6th (P6/K6)	1997	Pentium II / III, Celeron, AMD K6	64-bit	32 to 36-bit	4 to 64 GB
7th (NetBurst/K7)	1999	Pentium 4 / Pentium D, AMD Athlon	64-bit	36-bit	64 GB
8th (64-bit Core)	2003	Pentium Dual-Core, Core to Duo, AMD Athlon 64	64-bit	36 to 40-bit	64 GB to 1 TB
Modern (Core i)	2008+	Intel Core i3-i9 (1st–14th Gen)	64-bit	36 to 48-bit	128 GB to 2 TB
Latest (AI Era)	2024+	Intel Core Ultra, AMD Ryzen 9000, Ryzen AI	64-bit	48-bit	192 to 256 GB

*Note: Please study by yourself for details of each generations.

Basic Block Diagram of a Computer

- Computer contains four components: CPU (ALU+CU), Memory, Input and Output devices.
- The CPU contains various registers to store data, the ALU to perform arithmetic and logical operations, instruction decoders, counters and control lines.
- The CPU reads instructions from memory and performs the tasks specified.
- It communicates with input/output (I/O) devices either to accept or send data, the I/O devices is known as peripherals.



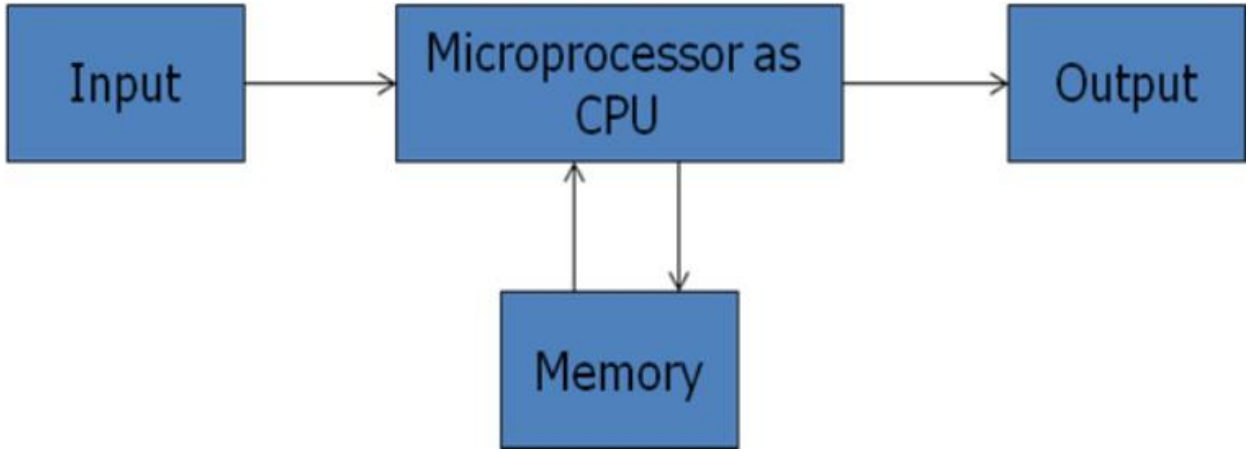
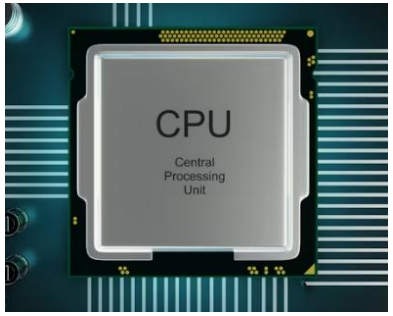


Fig: Block Diagram of a computer with the Microprocessor as CPU

- Around late 1960's, the traditional block diagram was replaced where microprocessor was used as CPU and the computer is known as microcomputer.
- The CPU was built using Integrated Circuit Technology (IC) which changed the CPU on a single chip.



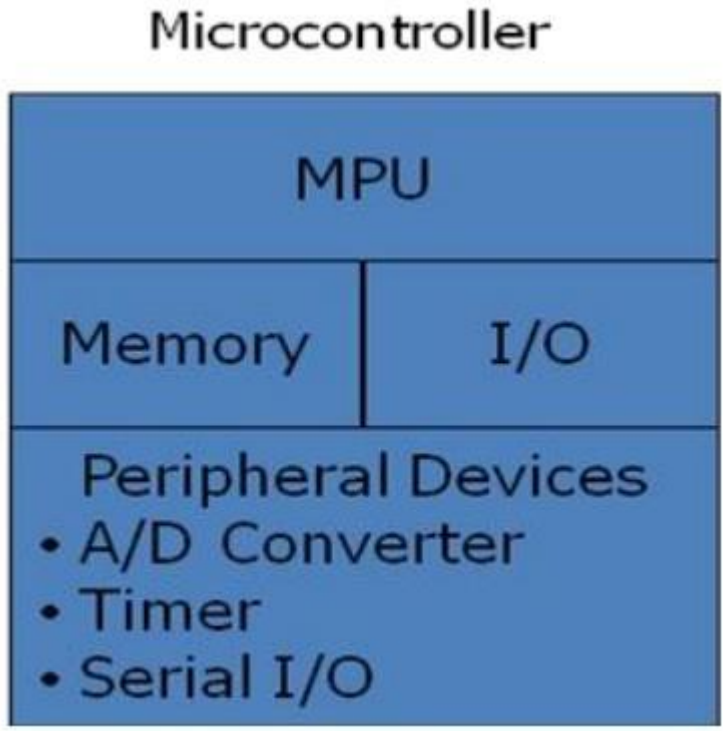


Fig: Block Diagram of a Microcontroller

Organization of a microprocessor based system

- Microprocessor based system includes three components: microprocessor, input/output and memory (read only and read/write).
- These components are organized around a common communication path called a bus.

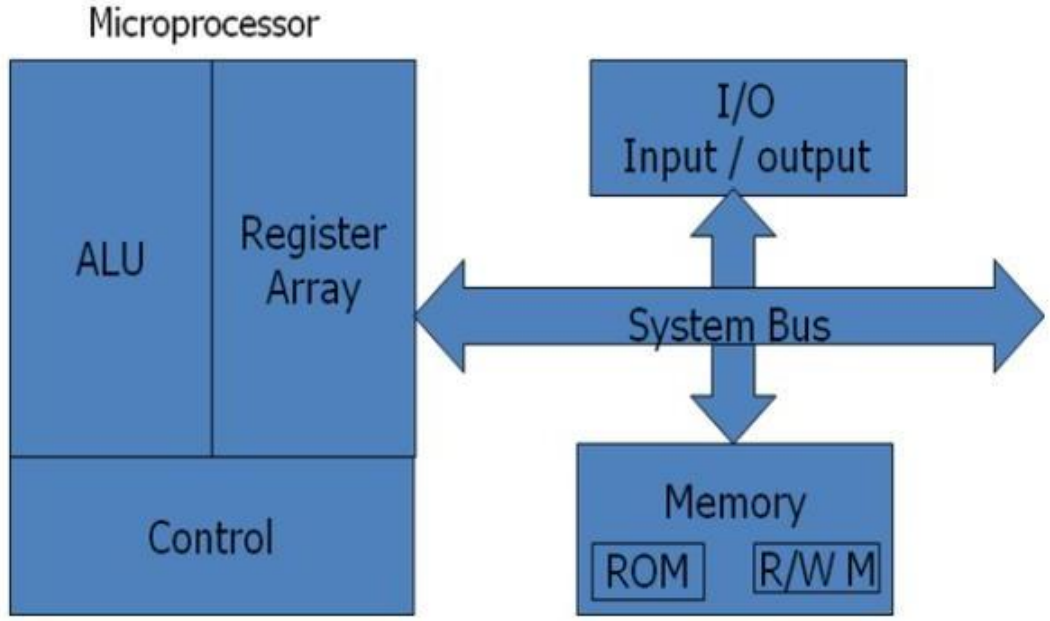


Fig: Microprocessor Based System with Bus Architecture

1. Microprocessor

A. Arithmetic/Logic unit
   - It performs arithmetic operations as addition and subtraction and logic operations as AND, OR & XOR.

B. Register array
   - The registers are primarily used to store data temporarily during the execution of a program and are accessible to the user through instruction.
   - The registers can be identified by letters such as B, C, D, E, H and L.

C. Control unit
   - It provides the necessary timing and control signals to all the operations in the microcomputer.
   - It controls the flow of data between the microprocessor and memory & peripherals.

Registers:

- Small, fast, internal storage units used to hold data, instruction addresses, and intermediate results temporarily.
- **General Purpose Registers (GPRs):** Used to store data (B, C, D, E, H, L) in 8085).
- **Special Purpose Registers:** Include the Program Counter (PC) to store the address of the next instruction, the Stack Pointer (SP), and the Instruction Register (IR).
- **Flag Register:** Stores status bits (flags) indicating the outcome of ALU operations (e.g., zero, carry).

Control and Timing Unit:

- This unit interprets instructions fetched from memory, decoding them to generate control signals that manage the internal and external operations of the microprocessor.
- It synchronizes all microprocessor operations with the system clock.

2. Memory

- Memory stores binary information such as instructions and data, and provides that information to the up whenever necessary.
- To execute programs, the microprocessor reads instructions and data from memory and performs the computing operations in its ALU. Results are either transferred to the output section for display or stored in memory for later use. Memory has two sections.

A. Read only Memory (ROM):

- Used to store programs that do not need alterations and can only read.

B. Read/Write Memory (RAM):

- Also known as user memory which is used to store user programs and data.
- The information stored in this memory can be easily read and altered.

3. Input/Output

- Microprocessor communicates with the outside world using two devices input and output which are also Known as peripherals.
- The input device such as keyboard, switches, and analog to digital converter transfer binary information from outside world to the microprocessor.
- The output devices transfer data from the microprocessor to the outside world. They include the devices such as LED, CRT, digital to analog converter, printer etc.

4. System Bus

- It is a communication path between the microprocessor and peripherals; it is nothing but a group of wires to carry bits.

- A system bus is a central set of parallel wires connecting a microprocessor (CPU) to memory and I/O devices, acting as the communication highway for data, addresses, and control signals.

- It consists of three main components: the Data Bus (bidirectional), Address Bus (unidirectional, determines location), and Control Bus (transmits commands).

Bus Organization

- Bus is a common channel through which bits from any sources can be transferred to the destination.
- A typical digital computer has many registers and paths must be provided to transfer instructions from one register to another.
- The number of wires will be excessive if separate lines are used between each register and all other registers in the system.
- A more efficient scheme for transferring information between registers in a multiple register configuration is a common bus system.
- A bus structure consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time.
- Control signals determine which register is selected by the bus during each particular register transfer.

- A very easy way of constructing a common bus system is with multiplexers.
- The multiplexers select the source register whose binary information is then placed on the bus.
- A system bus consists of about 50 to 100 of separate lines each assigned a particular meaning or function.
- Although there are many different bus designers, on any bus, the lines can be classified into three functional groups; data, address and control lines.
- In addition, there may be power distribution lines as well.

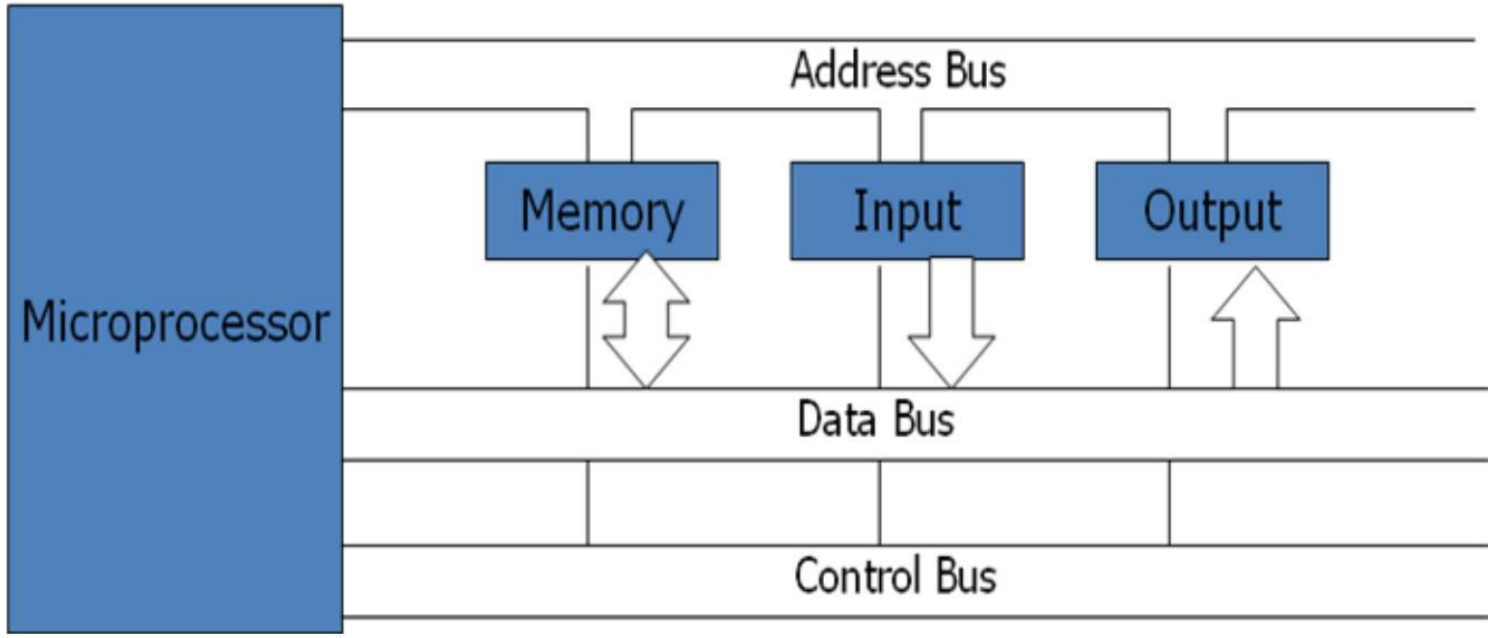


Fig: Bus Organization

- The **data lines** provide a path for moving data between system modules. These lines are collectively called data bus.
- The **address lines** are used to designate the source/destination of data on data bus.
- The **control lines** are used to control the access to and the use of the data and address lines.
  - Because data and address lines are shared by all components, there must be a means of controlling their use.
  - Control signals transmit both command and timing signals indicate the validity of data and address information.
  - Command signals specify operations to be performed.
  - Control lines include memory read/write, i/o read/write, bus request/grant, clock, reset, interrupt request/acknowledge etc.

Types of computers historically:

- **Fixed Program Computers** - Their function is very specific, and they couldn't be reprogrammed, e.g., calculators.

- **Stored Program Computers** - These can be programmed to carry out many different tasks; applications are stored on them, hence the name. eg. smartphones, laptops.

Stored Program Concept

- The stored program concept, introduced in the 1940s, is a foundational computing principle where computer instructions and data are stored together in the same main memory.
- This allows computers to switch between different applications without hardware reconfiguration, enabling versatile, general-purpose computing via the **fetch-decode-execute** cycle.
- Rather than rewiring computer for each new task, in the stored program concept the program/instructions are stored in memory along side the data.
- The program could be programmed or altered by changing the binary values.
- This concept gives rise to sequential program execution becoming software programmable rather than hardware programmable.
- This approach is known as 'stored-program concept' was first adopted by **John Von Neumann** and such architecture is named as **von-Neumann architecture.**

Von-Neumann Machine

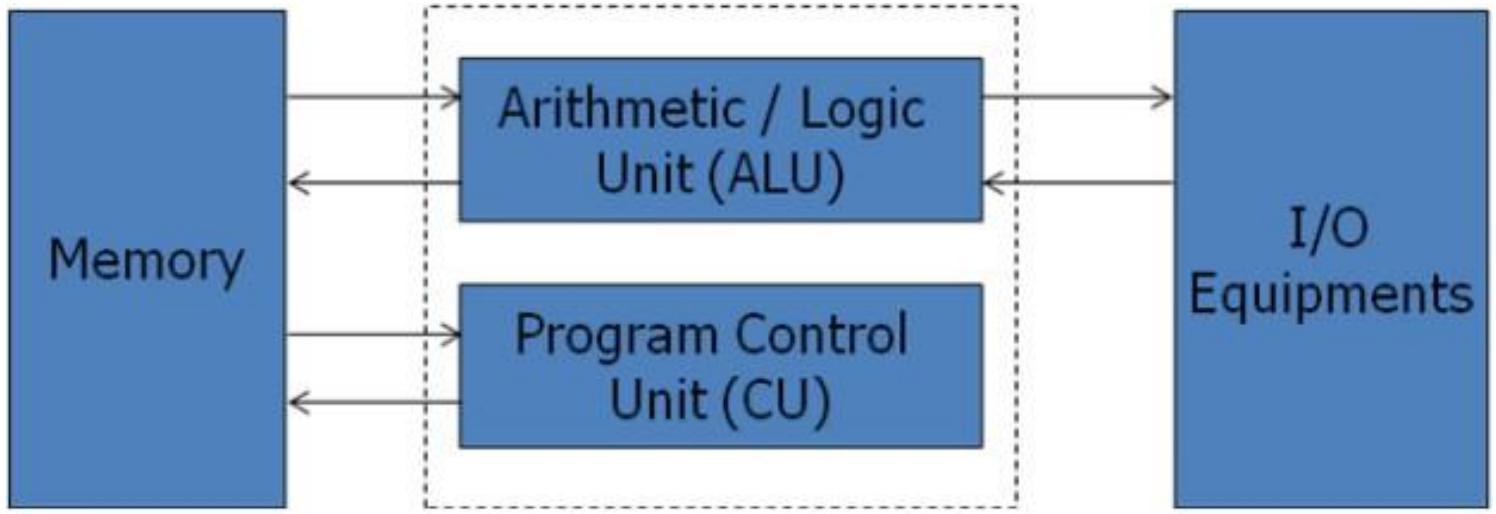


Fig: Von –Neumann Architecture

- The stored program concept enabled the development of flexible, programmable computers, separating hardware design from software functionality

Von-Neumann Machine

- The **main memory** is used to store both data and instructions.
- The **arithmetic and logic unit** is capable of performing arithmetic and logical operations on binary data.
- The program **control unit** interprets the instruction in memory and causes them to be executed.
- The **I/O** unit gets operated from the control unit.
- The Von–Neumann architecture is the fundamental basis for the architecture of modern digital computers.
- Initial von neumann machine (1945), consisted of 1024 storage locations (words) which can hold words of each 44 bits and both instructions as well as data are stored in it.
- The IAS machine (Institute for Advanced Study), which was the first concrete implementation of the Von Neumann architecture, had a similar capacity of 4,096 memory locations, with each word consisting of 40 bits.

- The storage location of control unit and ALU are called registers and there are various models of registers.
- Small block in the CPU that consists of a high-speed storage memory cells that store data before it is processed, all logical, arithmetic, and shift operations occur here.

- MAR – Memory Address Register – contains the address in memory of the word to be written into or read from MBR.
- MBR – Memory Buffer Register – consists of a word to be stored in or received from memory. Also called MDR-Memory Data Register
- IR – instruction register – contains the 8-bit op-code instruction to be executed.
- IBR – instruction buffer register – used to temporarily hold the instruction from a word in memory.
- PC - program counter - contains the address of the next instruction to be fetched from memory.
- AC & MQ (Accumulator and Multiplier Quotient) - holds the operands and results of ALU after processing.

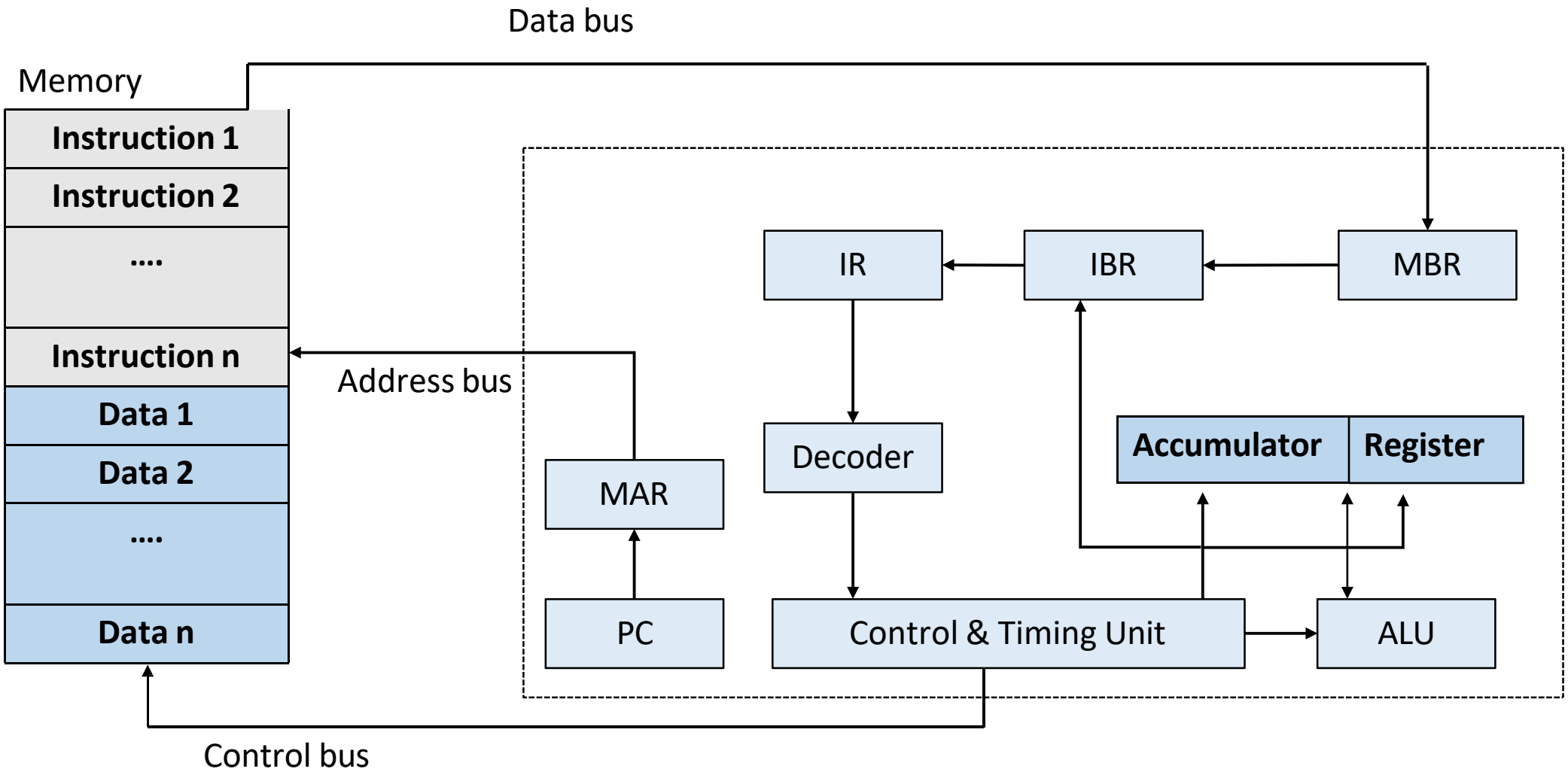


Fig: Structure of Von-Neuman's Machine or IAS Computer

Key Characteristics:

- **Single Memory for Data and Instructions:** Both data and program instructions are stored in the same memory.
- **Shared Bus:** A single bus is used for transferring data, addresses, and control signals, which can limit performance.
- **Sequential Execution:** Instructions are executed one at a time in a sequential manner.

Von Neumann bottleneck:

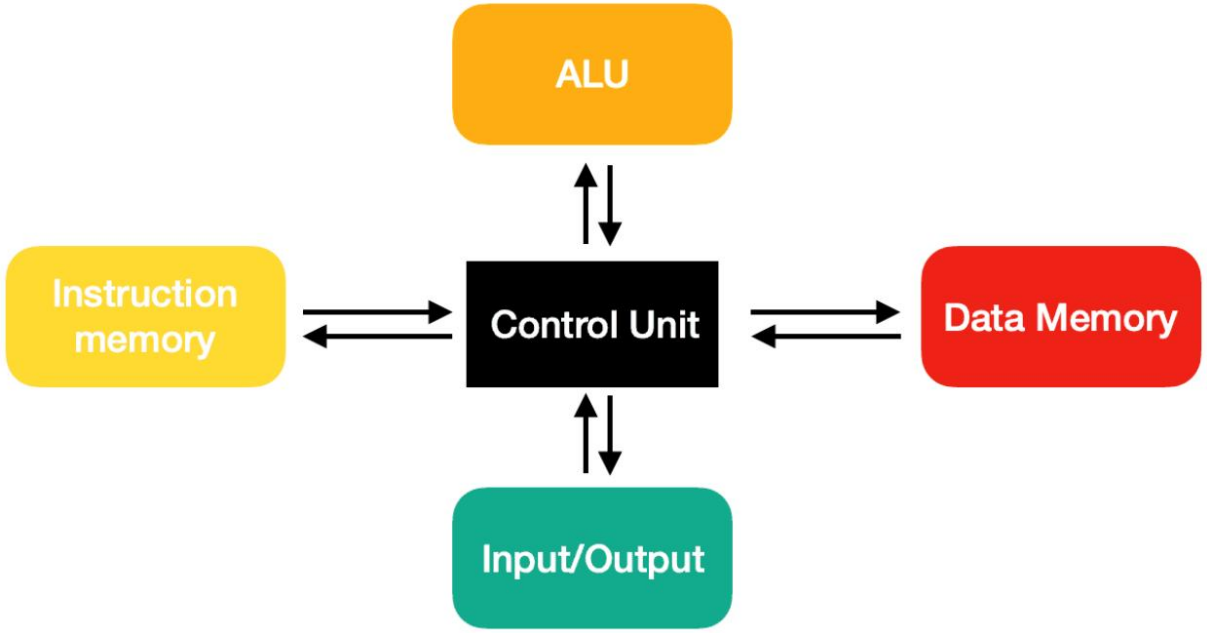
- Instructions can only be done one at a time and can only be carried out sequentially.
- The CPU spends a great amount of time being idle (doing nothing), while waiting for data to be fetched from the memory.
- No matter how fast the processor is, this ultimately depends on the rate of transfer, as a matter of fact, if the processor is faster, this just means that it'll have a greater "idle" time.

Harvard Architecture

- Harvard architecture is named after the “Harvard Mark I” relay-based computer, which was an IBM computer in the University of Harvard.
- The computer-stored instructions on “punched tape” (24 bits wide), furthermore the data was stored in electro mechanical counters.
- The CPU of these early computer systems contained the data storage entirely, and it provided no access to the instruction storage as data.
- Harvard architecture is a type of architecture, which stores the data and instructions separately, therefore splitting the memory unit. Also having its own data and address bus.

Harvard Architecture

- The CPU in a Harvard architecture system is enabled to fetch data and instructions simultaneously, due to the architecture having separate buses for data transfers and instruction fetches.



Harvard Architecture

Harvard Architecture

- Concurrent fetching of instruction and data from memory is possible, increasing the processing speed.
- It is used in microcontroller.

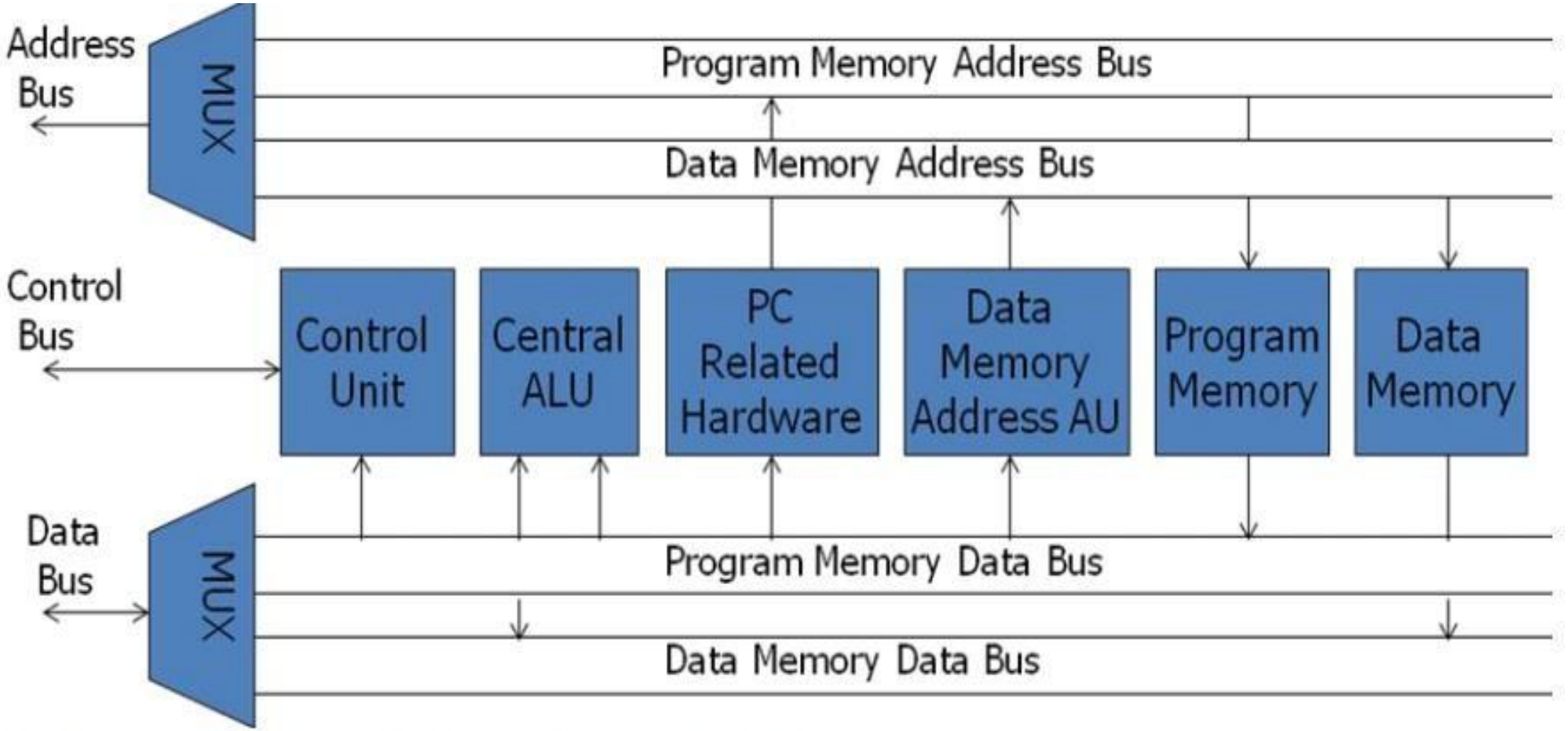


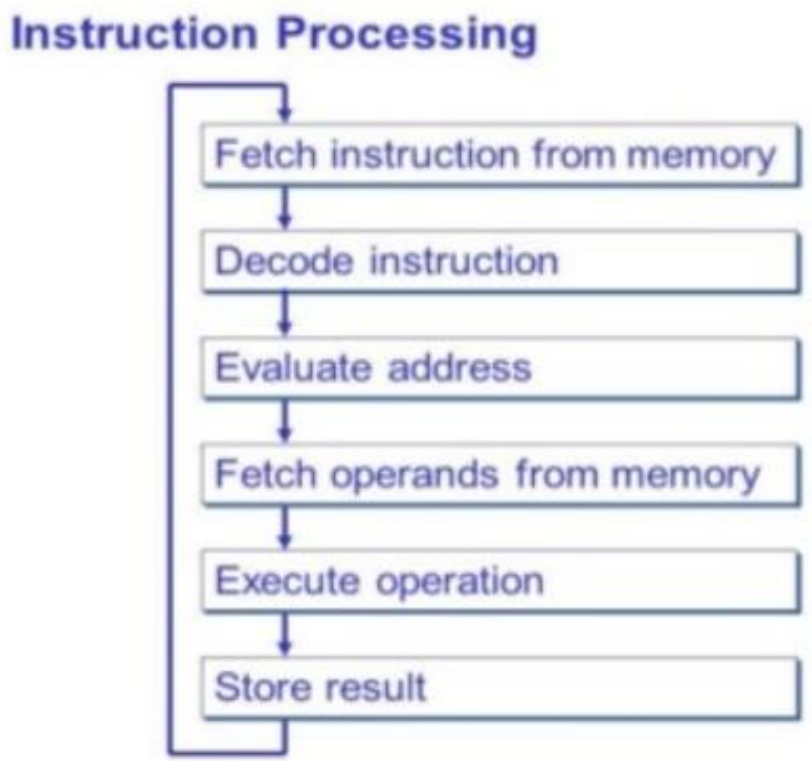
Fig: Harvard Architecture Based Microprocessor

Von Neumann Architecture vs. Harvard Architecture:

Von Neumann Architecture	Harvard Architecture
Based on the stored-program computer concept	Based on the Harvard Mark I relay-based computer model
Uses the same physical memory address for instructions and data	It uses separate memory addresses for instructions and data
The processors require two clock cycles to execute an instruction	Processor requires only one cycle to complete an instruction
The von Neumann architecture consists of a simpler control unit design, which means less complex development is required. This means the system will be less costly	Harvard architectures control unit consists of two buses, which results in a more complicated system. This adds to the development cost, resulting in a more expensive system
Instruction fetches and data transfers cannot be performed at the same time	Instruction fetches and data transfers can be performed at the same time
Used in laptops, personal computers, and workstations	Used in signal processing and micro-controllers

Stored Program Concept (operation)

- PC holds the address of recent executing instruction.
- MAR selects the particular memory location and the instruction is loaded to IR and PC is increased by One.
- The decoder decodes the instruction to control and timing unit
- The processing occurs at ALU and results are stored in appropriate registers.



Key Operation Steps (Fetch-Decode-Execute Cycle):

1. **Fetch (Get Instruction):** The CPU fetches the next instruction from the main memory (RAM) based on the address held in the Program Counter (PC).
2. **Decode (Interpret Instruction):** The Control Unit decodes the instruction inside the Instruction Register (IR) to determine what operation needs to be performed.
3. **Execute (Perform Operation):** The Arithmetic Logic Unit (ALU) or control unit executes the operation, such as adding numbers, loading data, or comparing values.
4. **Store (Store Result):** The results of the execution are written back into the memory or registers.
5. **Update PC (Program Counter):** The Program Counter is updated (incremented) to point to the next instruction in the sequence, allowing for continuous execution.

Processing Cycle of a Stored Program Computer (Instruction Cycle)

- The processing cycle of a stored program computer, known as the **fetch-decode-execute cycle**, is the fundamental operation where the CPU continuously fetches instructions from memory, decodes them, and executes them.
- The processing cycle is the time required to complete the execution of one instruction.
- Each cycle consist of two sub-cycles: Fetch-Decode and Execute

1. Fetch-Decode
   - It fetches the instruction code from memory and decodes them.
2. Execute
   - It executes the instruction which consist of calculating the address of operands, fetching them, performing operations on them and outputting the results to a specified location.

Key Stages of the Instruction Cycle

1. Fetch (Instruction Fetch): The CPU fetches the next instruction from memory. The Program Counter (PC) holds the address, which is copied to the Memory Address Register (MAR), and the instruction is transferred to the Instruction Register (IR). The PC is then incremented.

2. Decode: The Control Unit (CU) interprets the opcode in the instruction register (IR) to determine which operation is required (e.g., ADD, SUB, LOAD).

3. Operand Fetch/Read Address: If the instruction requires data from memory, the effective address is calculated, and the operand is fetched from that address. For simple register-based instructions, this step might be skipped.

4. Execute: The CPU performs the actual operation, such as an arithmetic calculation (ALU), data transfer, or logical operation.

5. Store (Writeback): The result generated by the execution is written back to memory or stored in a CPU register.

Instruction Cycle

- Once one instruction cycle is completed, the cycle repeats itself beginning with a new instruction phase cycle.
- Eg. MOV A,B

A. Fetch Decode Cycle

- Before microprocessor begins to execute the instruction it needs to fetch the code from memory where it is stored.

1. PC contains address of the next instruction to be executed. The content of PC is transferred to MAR.

T1: MAR ← PC

Fetch Decode Cycle

2. When control unit issues memory read signal, the content of memory locations specified by memory address register (MAR) is transfer into MBR.

T2: MBR ← [MAR]

3. Finally the content of MBR is transfer to IR and PC gets incremented by 1.

T3: IR ← MBR , PC← PC+1

Here IR contains the instruction code for execution which is then decoded after fetching

B. Execute Cycle

1. The address of B is transferred to MAR then to address bus.

T1: MAR ← IR (address of B)

2. When control unit issues memory read signal, the content of B is transferred to MBR.

T2: MBR ← B

3. The address of A from the address field IR is transferred to MAR.

T3: MAR ← IR (Address of A)

4. When control unit issues memory write signal the contents of MBR is transferred to the memory location specified by MAR which is A in this case.

T4: A ← MBR

In the above case, the elementary operations performed on the information stored in one or more register within single unit of time is called micro-operation. These are smaller blocks of fetch decode and execute cycle.

Introduction to Register Transfer Language

- RTL is the systematic notation used to describe the main operation transfer among registers.

RTL for MOV A, B:

Fetch Cycle:

T1: MAR <- PC

T2: MBR <- [MAR]

T3: IR <- MBR

PC <- PC + 1

Execute Cycle:

T4: MAR <- (IR (ADDRESS OF B))

T5: MAR <- [B]

T6: MAR <- (IR (ADDRESS OF A))

T7: A <- MAR

RTL for MVI A, 02H:

Fetch Cycle:

T1: MAR <- PC

T2: MBR <- [MAR]

T3: IR <- MBR

PC <- PC + 1

Execute Cycle:

T4: MBR <- (IR (ADDRESS OF IMMEDIATE DATA))

T5: MAR <- (IR (ADDRESS OF A))

T6: [MAR] <- MBR

RTL for LXI D, 9050H:

Fetch Cycle:

T1: MAR <- PC

T2: MBR <- [MAR]

T3: IR <- MBR

PC <- PC + 1

Execute Cycle:

T4: MBR <- IR [Address of immediate data (LSB)]

T5: MAR <- IR [Address of E]

T6: E <- MBR

T7: MBR <- IR [Address of immediate data (MSB)]

T8: MAR <- IR [Address of D]

T6: D <- MBR

Microinstructions

- Microinstructions are fundamental, low-level commands stored in a CPU's control memory (ROM) that orchestrate hardware operations, such as transferring data between registers, initiating ALU functions, or managing memory access, to execute complex machine instructions.
- Each instruction is characterized with many machine cycles and each cycle is characterized with many T-states.
- Microinstructions are low-level control words stored in a microprocessor's internal control memory that define the basic operations (micro-operations) for one clock cycle.
- The lower instruction level patterns which are the numerous sequences for a single instruction are known as microinstructions.

Examples of Micro-operations

- **Data Movement:** (DR <- M[AR]) (loading memory operand into Data Register).
- **Arithmetic/Logic:** (AC <- AC+DR) (adding data register content to accumulator).
- **Control Flow:** (AR<- PC) (transferring program counter to address register).

Control Unit

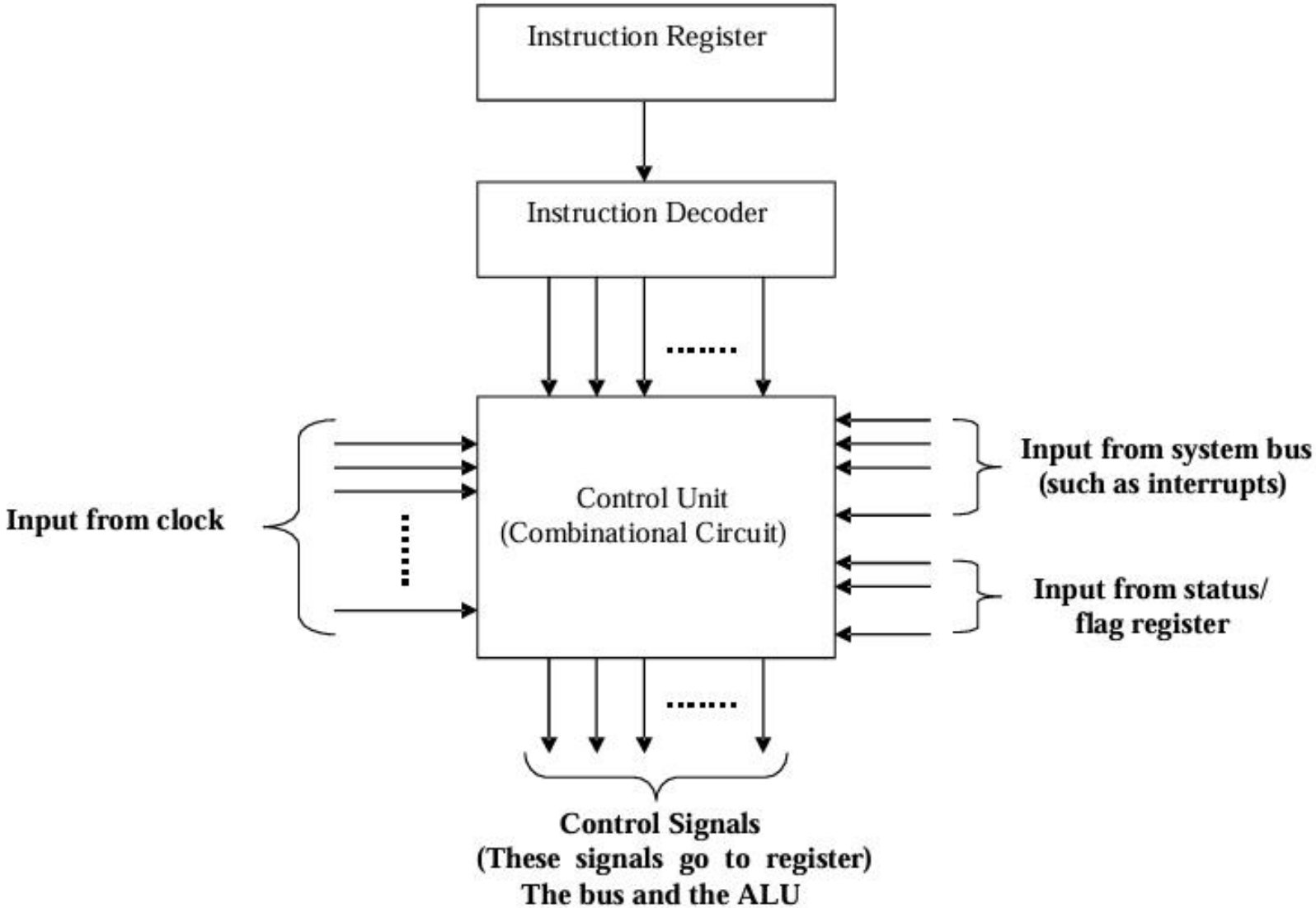
1. Hardwired Control Unit

The control unit essentially a state machine circuit design using conventional logic circuit design.

- It's i/p logic signals are transformed into set of o/p logic signals which are control signals.
- The CU performs different operations in the basis of op-codes.
- We have to derive the Boolean expression for each control signal as a function of input.
- It uses fixed instruction, fixed logic blocks of logic gate arrays, encoder, decoder, etc.
- Since modern processor needs a Boolean equation, it is very difficult to build a combinational circuit that satisfies all these operations.
- It has faster mode of operation.
- A hardwired control unit needs rewiring if design has to be modified.

Examples: Intel 8085, Motorolla 6802 etc

- The control hardware can be viewed as a state machine that changes from one state to another in every clock cycle, depending on the contents of the instruction register, the condition codes, and the external inputs.
- The outputs of the state machine are the control signals.
- The sequence of the operation carried out by this machine is determined by the wiring of the logic elements and hence is named “hardwired”.
- Fixed logic circuits that correspond directly to the Boolean expressions are used to generate the control signals.
- Hardwired control is faster than microprogrammed control.
- A controller that uses this approach can operate at high speed.
- RISC architecture is based on the hardwired control unit



Hardwired Control Organization

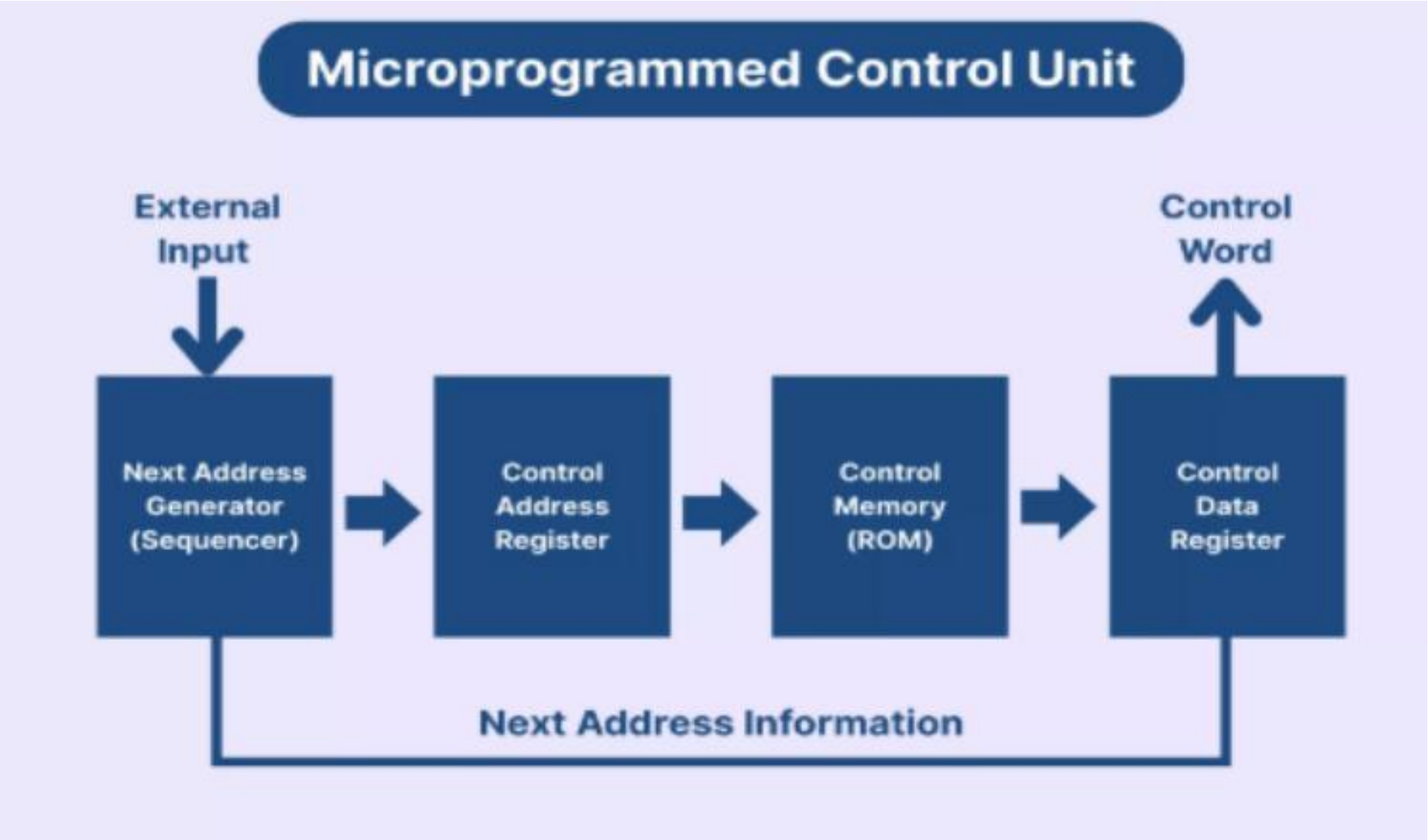
Control Unit

2. Micro-programmed Control Unit

An alternative to hardwired CU.

- In micro-programmed control unit, the control information is stored in control memory.
- The control memory is programmed to initiate required sequence of operations.
- Use sequences of instructions to perform control operations performed by micro operations.
- Control address register contains the address of the next micro instruction to be read.
- As it is read, it is transferred to control buffer register.
- Sequencing unit loads the control address register and issues a read command.
- It is cheaper and simple than hardwired CU.
- It is slower than hardwired CU.
- Examples: Intel Core i3, i7 (x86 architecture)-Hybrid (predominantly Microprogrammed with Hardwired acceleration), Motorola 68000-Microprogrammed, IBM Mainframes-Microprogrammed.

- The control signals associated with operations are stored in special memory units inaccessible by the programmer as Control Words.
- Control signals are generated by a program that is similar to machine language programs.
- The micro-programmed control unit is slower in speed because of the time it takes to fetch microinstructions from the control memory.


Microprogrammed Control Unit
External Input
Control Word
Next Address Generator (Sequencer)
Control Address Register
Control Memory (ROM)
Control Data Register
Next Address Information

Types of Micro-programmed Control Unit

1. **Horizontal Micro-programmed Control Unit :**

- The control signals are represented in the decoded binary format that is 1 bit/CS.
- Example: If 53 Control signals are present in the processor then 53 bits are required. More than 1 control signal can be enabled at a time.
- It supports longer control words.
- It is used in parallel processing applications.
- It allows a higher degree of parallelism. If degree is n, n CS is enabled at a time.
- It requires no additional hardware(decoders). It means it is faster than Vertical Microprogrammed.
- It is more flexible than vertical microprogrammed

2. Vertical Micro-programmed Control Unit :

- The control signals are represented in the encoded binary format. For N control signals- Log2(N) bits are required.
- It supports shorter control words.
- It supports easy implementation of new control signals therefore it is more flexible.
- It allows a low degree of parallelism i.e., the degree of parallelism is either 0 or 1.
- Requires additional hardware (decoders) to generate control signals, it implies it is slower than horizontal microprogrammed.
- It is less flexible than horizontal but more flexible than that of a hardwired control unit.

Hardwired Control Unit	Microprogrammed Control Unit
• Hardwired control unit generates the control signals needed for the processor using logic circuits	• Microprogrammed control unit generates the control signals with the help of micro instructions stored in control memory
• Hardwired control unit is faster when compared to microprogrammed control unit as the required control signals are generated with the help of hardwares	• This is slower than the other as micro instructions are used for generating signals here
• Difficult to modify as the control signals that need to be generated are hard wired	• Easy to modify as the modification need to be done only at the instruction level
• More costlier as everything has to be realized in terms of logic gates	• Less costlier than hardwired control as only micro instructions are used for generating control signals
• It cannot handle complex instructions as the circuit design for it becomes complex	• It can handle complex instructions
• Only limited number of instructions are used due to the hardware implementation	• Control signals for many instructions can be generated
• Used in computer that makes use of Reduced Instruction Set Computers (RISC)	• Used in computer that makes use of Complex Instruction Set Computers (CISC)

SAP1 Architecture (8 bit processor)

- The SAP-1 (Simple-As-Possible 1) is an educational 8-bit bus-organized microprocessor designed to demonstrate basic computer architecture.
- Developed by Dr. Albert Paul Malvino and Dr. Jerald A. Brown.

- It features a 16x8 RAM, a 4-bit ALU, 8-bit registers (accumulator, B register, instruction register), and a hardwired control unit that executes a limited instruction set (LDA, ADD, SUB, OUT, HLT) using 5 microsteps per instruction.
- 8-bit data bus, 4-bit address bus, 16 bytes maximum memory and have Hardwired Control Unit.

- SAP1 uses random access memory (RAM) for both data and instructions.
- The memory is organized into two sections: one for storing instructions and another for data.
- Programming for SAP1 is done using machine language instructions, and the system is manually controlled.

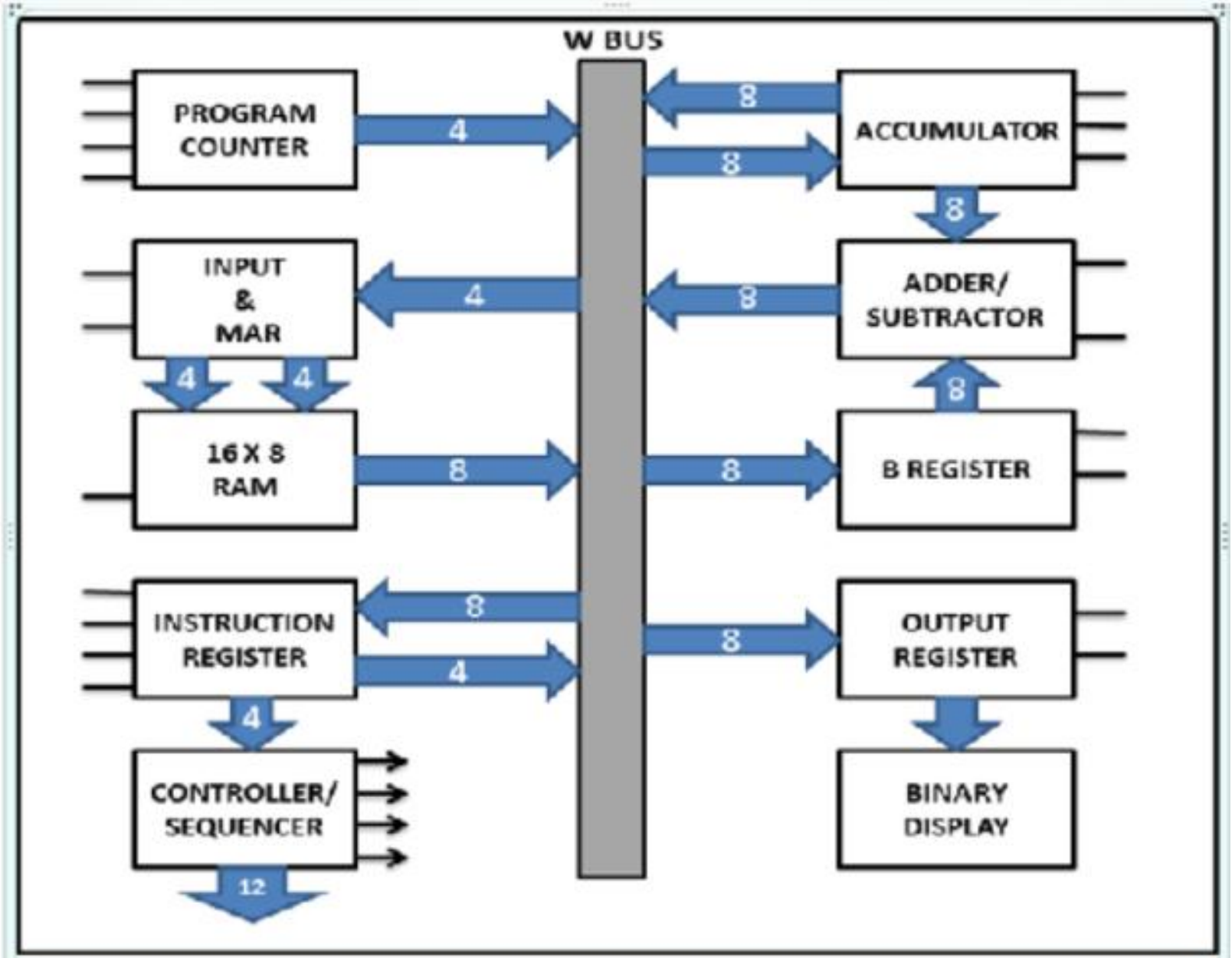


Fig (i) – SAP-1 Architecture

Core Components of SAP-1 Architecture

1. **W-Bus:** An 8-bit bidirectional bus that connects all components, allowing orderly data transfer using three-state buffers.
2. **Program Counter (PC):** A 4-bit register that holds the address of the next instruction to be fetched.
3. **Input and Memory Address Register (MAR):** A 4-bit register that receives the address from the PC or IR to access the 16x8 RAM.
4. **RAM (16x8):** Stores both programs and data, with a capacity of 16 nibble-addressable locations.
5. **Instruction Register (IR):** An 8-bit register that holds the current instruction, which is split into a 4-bit opcode and 4-bit operand.
6. **Controller/Sequencer:** Generates 12 control signals (via a 6-state ring counter) to synchronize operations.
7. **Accumulator (A):** An 8-bit register that stores intermediate arithmetic/logic results.
8. **Adder/Subtractor:** A 4-bit circuit that computes (A + B) or (A - B).
9. **B Register:** An 8-bit register used for storing the second operand during addition or subtraction.
10. **Output Register:** An 8-bit register (often connected to LEDs) that displays the final result.

SAP-1 Operation

- SAP-1 operates in a 3-cycle process: Fetch, Decode, and Execute, using a relatively slow clock to show the step-by-step movement of data.

- **Fetch Cycle:** Moves the address from the Program Counter to the MAR, reads the instruction from RAM, and loads it into the Instruction Register.

- **Execute Cycle:** The control unit decodes the opcode (e.g., LDA, ADD) and generates signals to perform the operation, such as loading data into the accumulator or performing arithmetic.

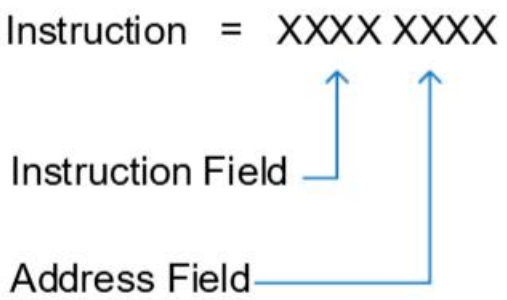
SAP-1 Instruction Set Summary

The instruction set is focused on basic data movement and arithmetic, utilizing a 16-byte memory capacity.

- **LDA (Load Accumulator):** 0000 – Loads data from a specific memory address into the accumulator.
- **ADD (Add):** 0001 - Adds the content of a specific memory address to the accumulator.
- **SUB (Subtract):** 0010 – Subtracts the content of a specific memory address from the accumulator.
- **OUT (Output):** 1110 – Transfers the accumulator content to the output register.
- **HLT (Halt):** 1111 – Stops the clock and terminates program execution.

Mnemonic	Operation	OPCODE
LDA	Load addressed memory contents into accumulator	0000
ADD	Add addressed memory contents to accumulator	0001
SUB	Subtract addressed memory contents from accumulator	0010
OUT	Load accumulator data into output register	1110
HLT	Stop processing	1111

Instruction Structure



- Total Width: 8 bits.
- The first four bits make the op-code while the last four bits make the operand (address).
- Opcode (4 bits): The top 4 bits (7-4) define the operation.
- Operand/Address (4 bits): The lower 4 bits (3-0) define the memory address ($0\text{-}F_{hex}$) to be used.

Example Instruction

- If LDA is defined as 0001 and we want to load data from address 9 (1001 in binary): then the Machine Code will be: 0001 1001 (19_{hex}).

Opcode and Mnemonics Table

Mnemonic	Op code
LDA	0000 (0H)
ADD	0001 (1H)
SUB	0010 (2H)
OUT	1110 (EH)
HLT	1111 (FH)

SAP 1 Program Example in Mnemonic form

Address	Mnemonics
0H	LDA 9H
1H	ADD AH
2H	ADD BH
3H	SUB CH
4H	OUT
5H	HLT

Memory-Reference Instructions:

- LDA, ADD, and SUB are called memory-reference instructions because they use data stored in the memory.
- OUT and HLT, on the other hand, are not memory-reference instructions because they do not involve data stored in the memory.

Machine cycle and Instruction cycle

- SAP1 has six T-states (three fetch and three execute cycles) reserved for each instruction.
- Not all instructions require all the six T-states for execution.
- The unused T- state is marked as No Operation (NOP) cycle.
- Each T-state is called a machine cycle for SAP1.
- A ring counter is used to generate a T-state at every falling edge of clock pulse. The ring counter output is reset after the 6th T-state.
- FETCH CYCLE – T1, T2, T3 machine cycle
- EXECUTE CYCLE - T4, T5, T6 machine cycle

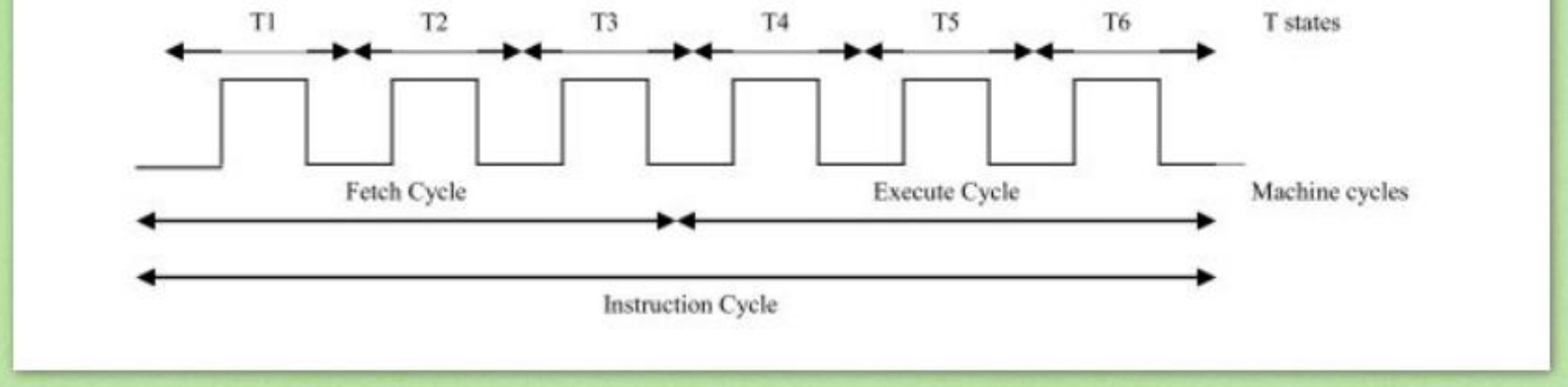


Fig: Instruction Cycle

- Complete code includes opcode and operand
- One instruction is executed in one instruction cycle
- Instruction cycle may consist of many machine cycles
- For SAP-1, Instruction cycle = Machine cycle
- Instruction cycle = Fetch cycle + Execution cycle
- Fetch cycle is generally same for all instructions
- Complete code includes opcode and operand
- Like LDA 04H 0000 0100
- One instruction is executed in one instruction cycle

Fetch and Execution Cycle of SAP-1 instructions

- SAP-1 instruction cycle: 3 clock cycles to fetch and decode phase, 3 clock cycles to execute
- The first three states are:
    1. address state
    2. increment state
    3. memory state
- Controller has a 6-bit ring counter which continuously cycles from 000001 up to 100000 then resets (must be set to 000001 when we initialize the computer)
- Ring counter is clocked on clock high-to-low transition (falling edge), most of the other circuits in the computer on clock low-to-high transition (rising edge)

Ring Counter

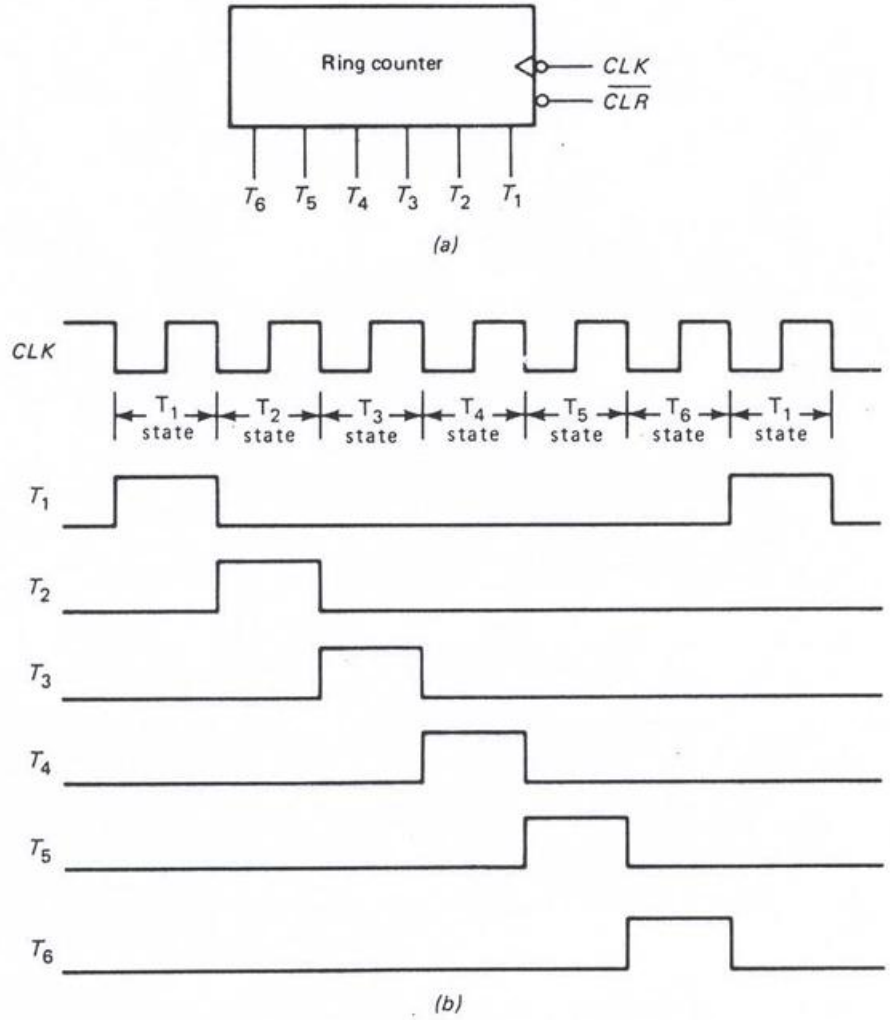
$$\mathbf{T} = T_6T_5T_4T_3T_2T_1$$

At the beginning of a computer run, the ring word is

$$\mathbf{T} = 000001$$

Successive clock pulses produce ring words of

$$\mathbf{T} = 000010$$
$$\mathbf{T} = 000100$$
$$\mathbf{T} = 001000$$
$$\mathbf{T} = 010000$$
$$\mathbf{T} = 100000$$



Ring counter: (*a*) symbol; (*b*) clock and timing signals.

Then, the ring counter resets to 000001, and the cycle repeats. Each ring word represents one *T* state.

Fetch Cycle

- Address state: enable PC to bus three-state output, MAR load line
- Increment state: enable PC increment (and perhaps wait for memory access time)
- Memory state: enable memory CE (Chip Enable), IR load line
- IR is loaded on the low-to-high clock transition, so stabilizes before state 4 is entered

t1: MAR ← PC
t2: PC ← PC +1
t3: IR ← RAM

Execution Cycle -- LDA

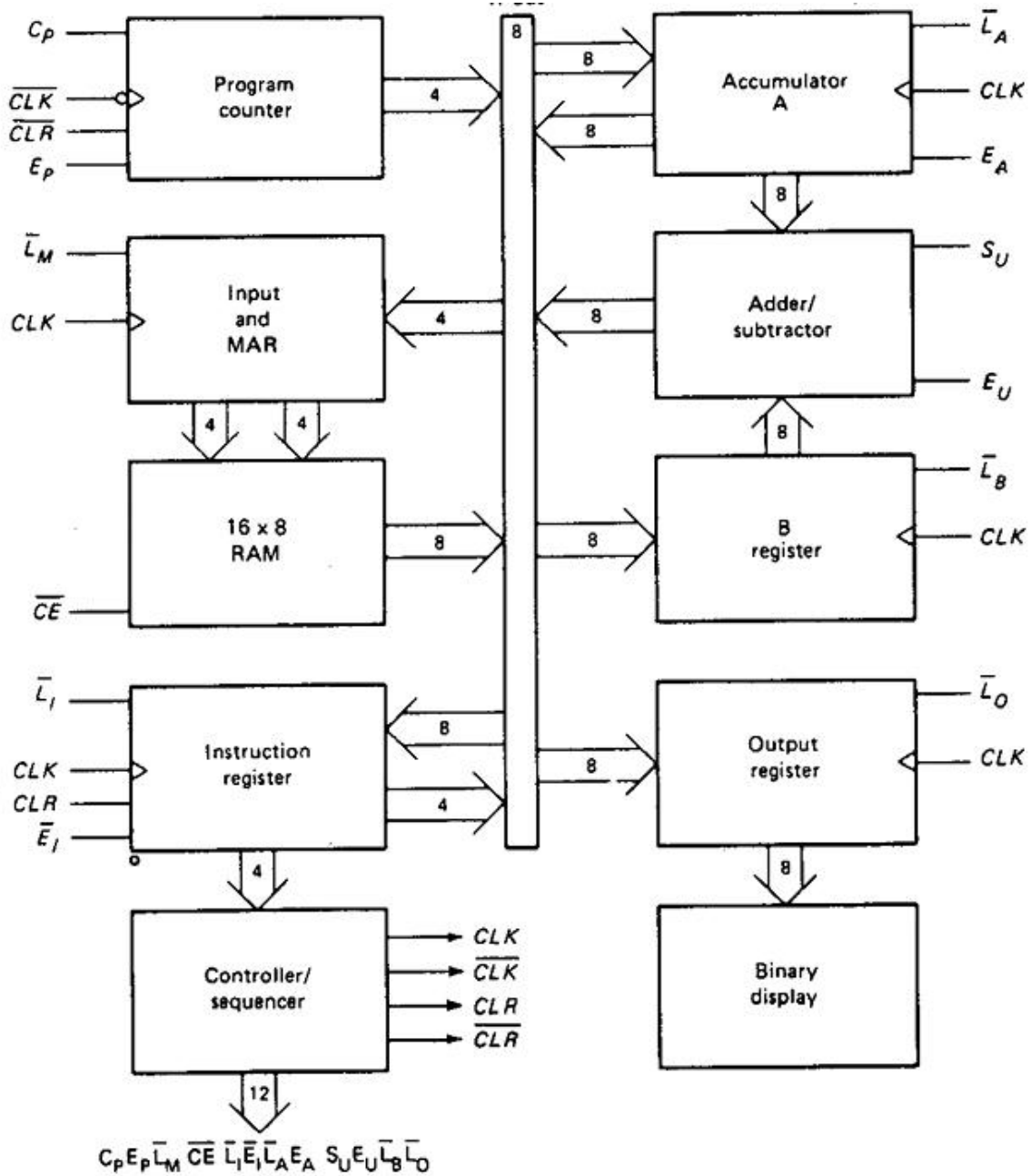
- state 4: enable IR to bus three-state output, MAR load line
- state 5: enable memory CE, accumulator load line
- state 6: enable nothing

t4: MAR ← (IR (Address of operand))
t5: Accumulator ← RAM
t6: nothing

Execution Cycle -- ADD

- state 4: enable IR to bus three-state output, MAR load line
- state 5: enable memory CE, register B load line
- state 6: enable add, ALU to bus three-state output, accumulator load line

t4: MAR ← (IR (Address of B))
t5: B ← RAM
t6: Accumulator ← Accumulator + B



SAP-1 architecture.

Control Word

- In SAP−1 (Simple−As−Possible 1) computer architecture, **C_P, E_P, L_M', CE', L_I', E_I', L_A', E_A, S_U, E_U, L_B', L_O'** are part of the 12−bit Control Word (CON) generated by the Controller−Sequencer to manage data flow and register operations on the W−Bus.

- Most of these signals are active−low (meaning they work when set to 0), while some are active−high (work at 1), and they synchronize with the positive edge of the clock (CLK).

- The full control word is typically:

CON = $C_P E_P L_M' CE' L_I' E_I' L_A' E_A S_U E_U L_B' L_O'$

$$C_P E_P \overline{L_M}\ \overline{CE}\ \overline{L_I}\,\overline{E_I}\,\overline{L_A} E_A\ S_U E_U \overline{L_B}\ \overline{L_O}$$

- **E_P (Enable Program Counter - Active High):** Connects the Program Counter (PC) to the W-Bus. When E_P is high, the address in the PC is placed onto the bus.
- **L_M (Load Memory Address Register - Active Low):** Loads the memory address from the W-Bus into the MAR.
- **C_P (Count Program Counter - Active High):** Increments the Program Counter (PC). When C_P is high, PC = PC + 1 on the next clock edge.
- **CE (Chip Enable/RAM Load - Active Low):** Enables the RAM to output data onto the bus. When active, it places the data from the memory location (indicated by MAR) onto the W-Bus.
- **L_I/L_1 (Load Instruction Register - Active Low):** Loads the instruction from the W-Bus into the Instruction Register (IR).

- **E_I/E_1 (Enable Instruction Register - Active Low):** Enables the lower 4 bits of the Instruction Register to output onto the W-Bus.
- **L_A (Load Accumulator - Active Low):** Loads data from the W-Bus into the Accumulator.
- **E_A (Enable Accumulator - Active High):** Places Accumulator data on the bus.
- **S_U (Substract - Active High):** Selects subtraction in ALU (if high, it subtracts; if low, it adds).
- **E_U (Enable User/Adder-Subtractor - Active High):** Places the result of the ALU onto the W-Bus.
- **L_B (Load B Register - Active Low):** Loads data from the W-Bus to the B Register.
- **L_O (Load Output Register - Active Low):** Loads data from the W-Bus to the Output Register.

Typical Example: Fetch Cycle (T1-T3)

- T1 (Address State): C_p (inactive), E_p (active), L_M (active), C_E (inactive) ...
- T2 (Increment State): C_p (active), E_p (inactive), L_M (inactive) ...
- T3 (Memory State): C_E (active), L_I (active) ...

Address state: 0101 1110 0011 (5E3H)

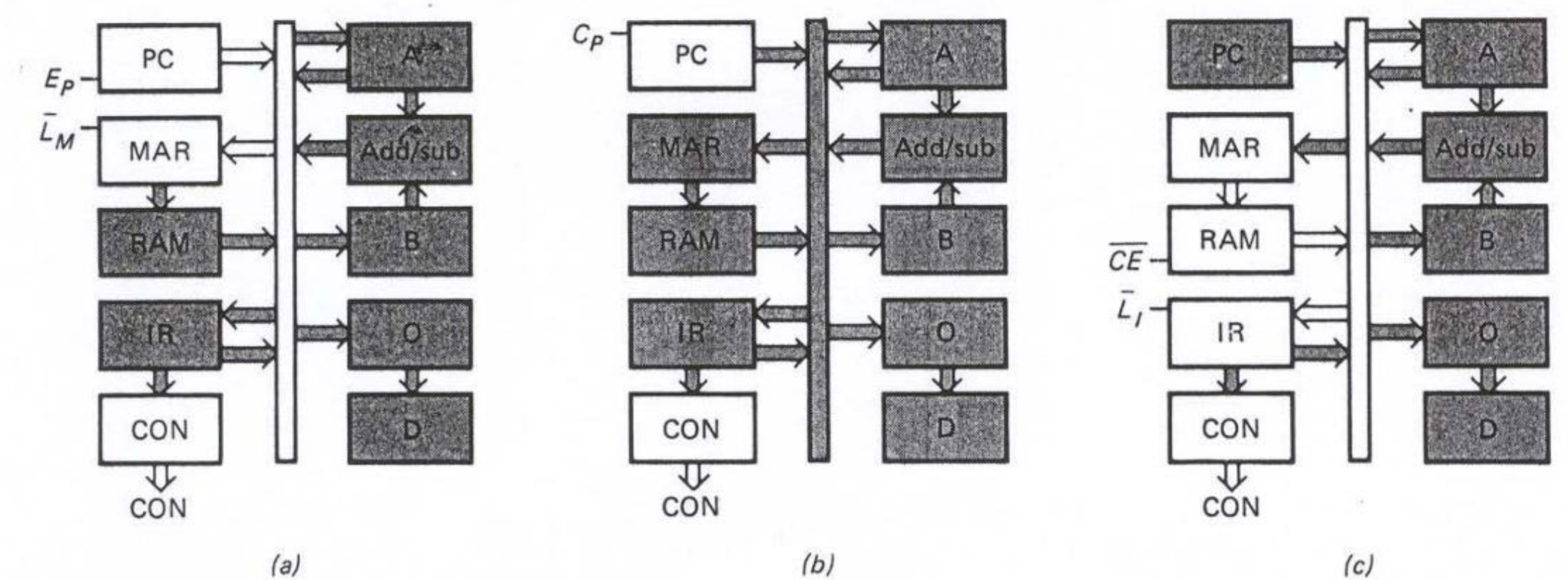
$$\begin{aligned}\text{CON} &= C_P E_P \bar{L}_M \overline{CE} \quad \bar{L}_I \bar{E}_I \bar{L}_A E_A \quad S_U E_U \bar{L}_B \bar{L}_O \\ &= 0\ 1\ 0\ 1 \quad 1\ 1\ 1\ 0 \quad 0\ 0\ 1\ 1\end{aligned}$$

Increment state:1011 1110 0011 (BE3H)

$$\begin{aligned}\text{CON} &= C_P E_P \bar{L}_M \overline{CE} \quad \bar{L}_I \bar{E}_I \bar{L}_A E_A \quad S_U E_U \bar{L}_B \bar{L}_O \\ &= 1\ 0\ 1\ 1 \quad 1\ 1\ 1\ 0 \quad 0\ 0\ 1\ 1\end{aligned}$$

Memory state: 0010 0110 0011 (263H)

$$\begin{aligned}\text{CON} &= C_P E_P \bar{L}_M \overline{CE} \quad \bar{L}_I \bar{E}_I \bar{L}_A E_A \quad S_U E_U \bar{L}_B \bar{L}_O \\ &= 0\ 0\ 1\ 0 \quad 0\ 1\ 1\ 0 \quad 0\ 0\ 1\ 1\end{aligned}$$

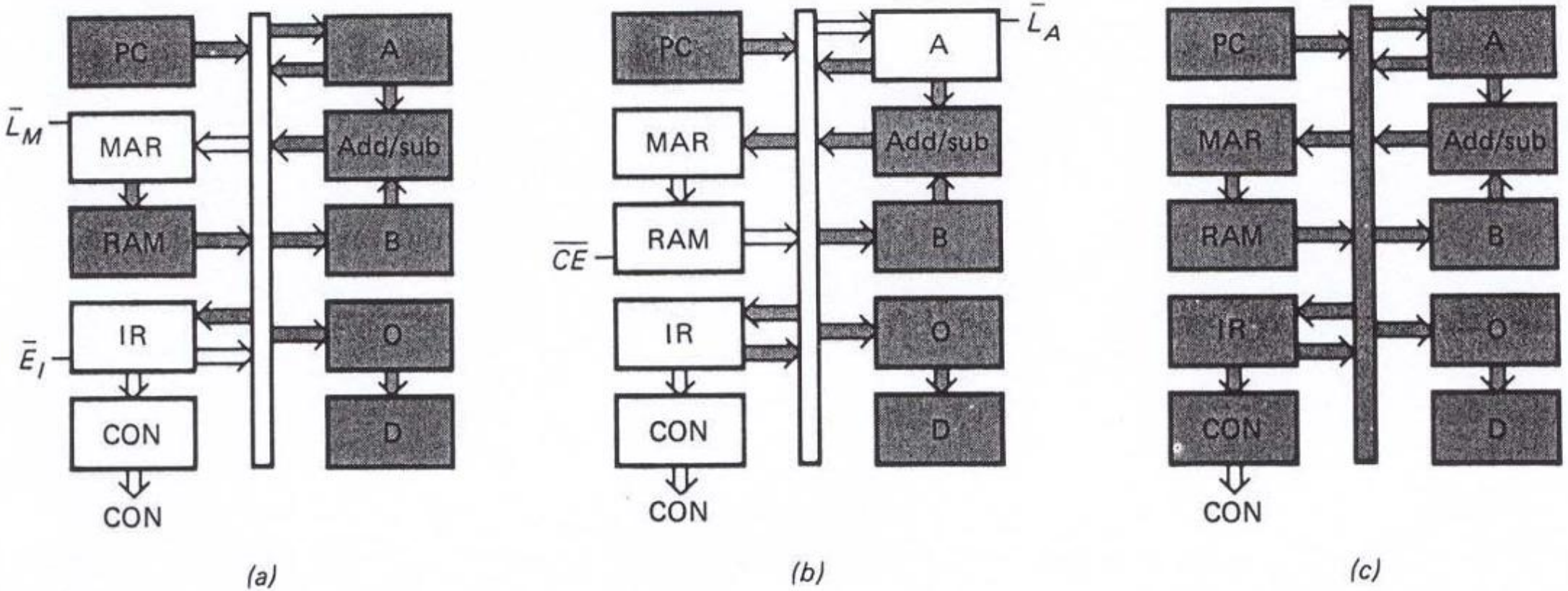


Fetch cycle: (*a*) T_1 state; (*b*) T_2 state; (*c*) T_3 state.

LDA Execution Cycle (T4–T6)

- These states move the operand (data) from the specified memory address into the Accumulator.

- **T4 (Operand Address to MAR):** The lower 4 bits of the IR (address field) are sent to the MAR.
  - **Active Bits:** $\overline{E}_I$, $\overline{L}_M$
  - **Control Word:** `0001 1010 0011` (1A3H)
- **T5 (RAM Data to Accumulator):** Data from the RAM address (now in MAR) is moved to the Accumulator.
  - **Active Bits:** $\overline{C}_E$, $\overline{L}_A$
  - **Control Word:** `0010 1100 0011` (2C3H)
- **T6 (No Operation):** All registers are inactive for this instruction.
  - **Active Bits:** None (all inactive)
  - **Control Word:** `0011 1110 0011` (3E3H)



LDA routine: (*a*) T_4 state; (*b*) T_5 state; (*c*) T_6 state.

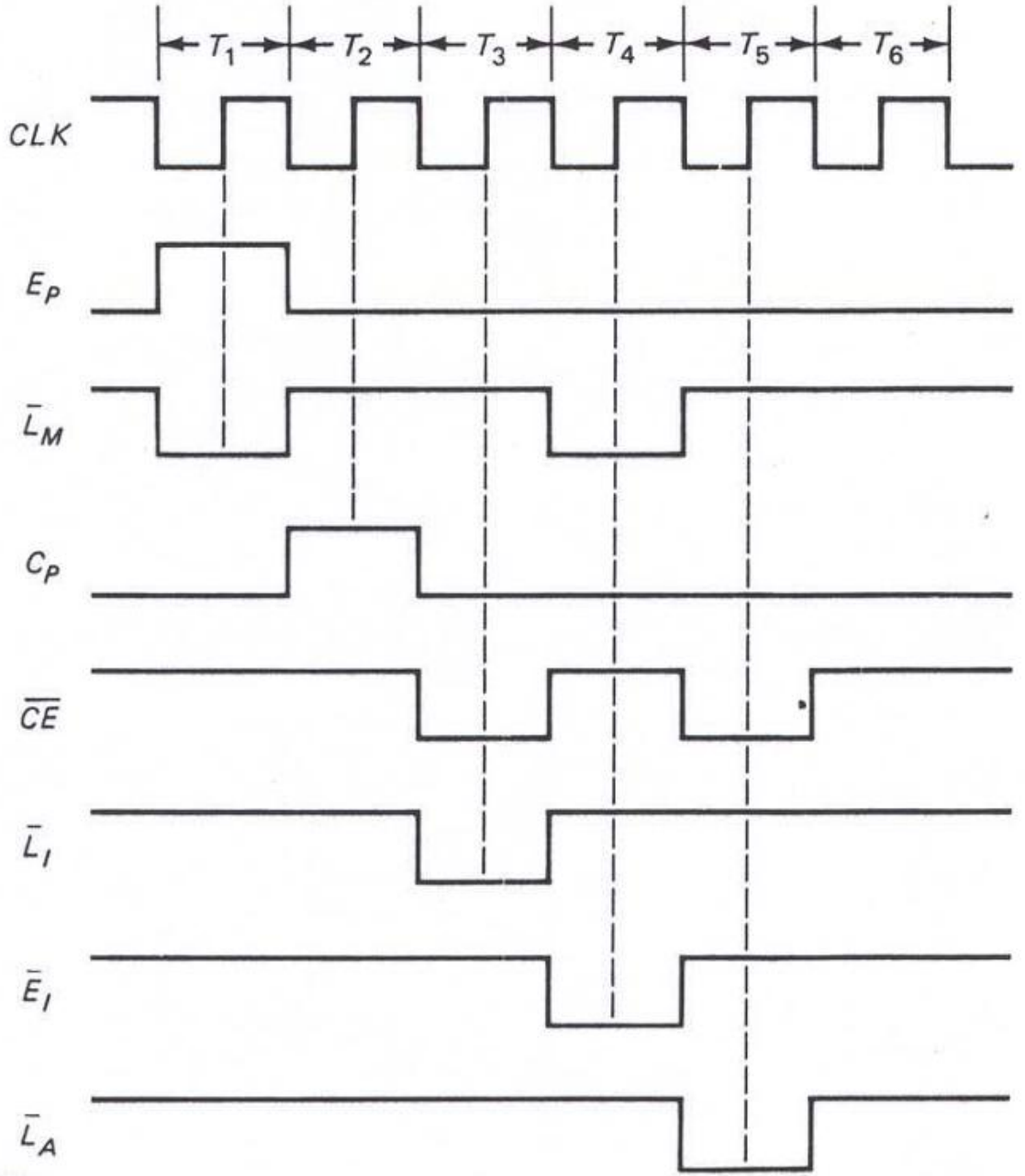
Summary Table for LDA

$$CON = C_P E_P L_M' C_E' L_I' E_I' L_A' E_A S_U E_U L_B' L_O'$$

State	Operation	Active Control Bits	Hex Word
T1	$MAR \leftarrow PC$	$E_P, \overline{L}_M$	5E3H
T2	$PC \leftarrow PC + 1$	C_P	BE3H
T3	$IR \leftarrow RAM[MAR]$	$\overline{C}_E, \overline{L}_I$	263H
T4	$MAR \leftarrow IR[0-3]$	$\overline{E}_I, \overline{L}_M$	1A3H
T5	$ACC \leftarrow RAM[MAR]$	$\overline{C}_E, \overline{L}_A$	2C3H
T6	NOP	None	3E3H

$$CON = C_P E_P L_M' C_E' L_I' E_I' L_A' E_A S_U E_U L_B' L_O'$$

State	Operation	Active Control Bits
T1	$MAR \leftarrow PC$	$E_P, \overline{L}_M$
T2	$PC \leftarrow PC + 1$	C_P
T3	$IR \leftarrow RAM[MAR]$	$\overline{C}_E, \overline{L}_I$
T4	$MAR \leftarrow IR[0-3]$	$\overline{E}_I, \overline{L}_M$
T5	$ACC \leftarrow RAM[MAR]$	$\overline{C}_E, \overline{L}_A$
T6	NOP	None



Fetch and LDA timing diagram.

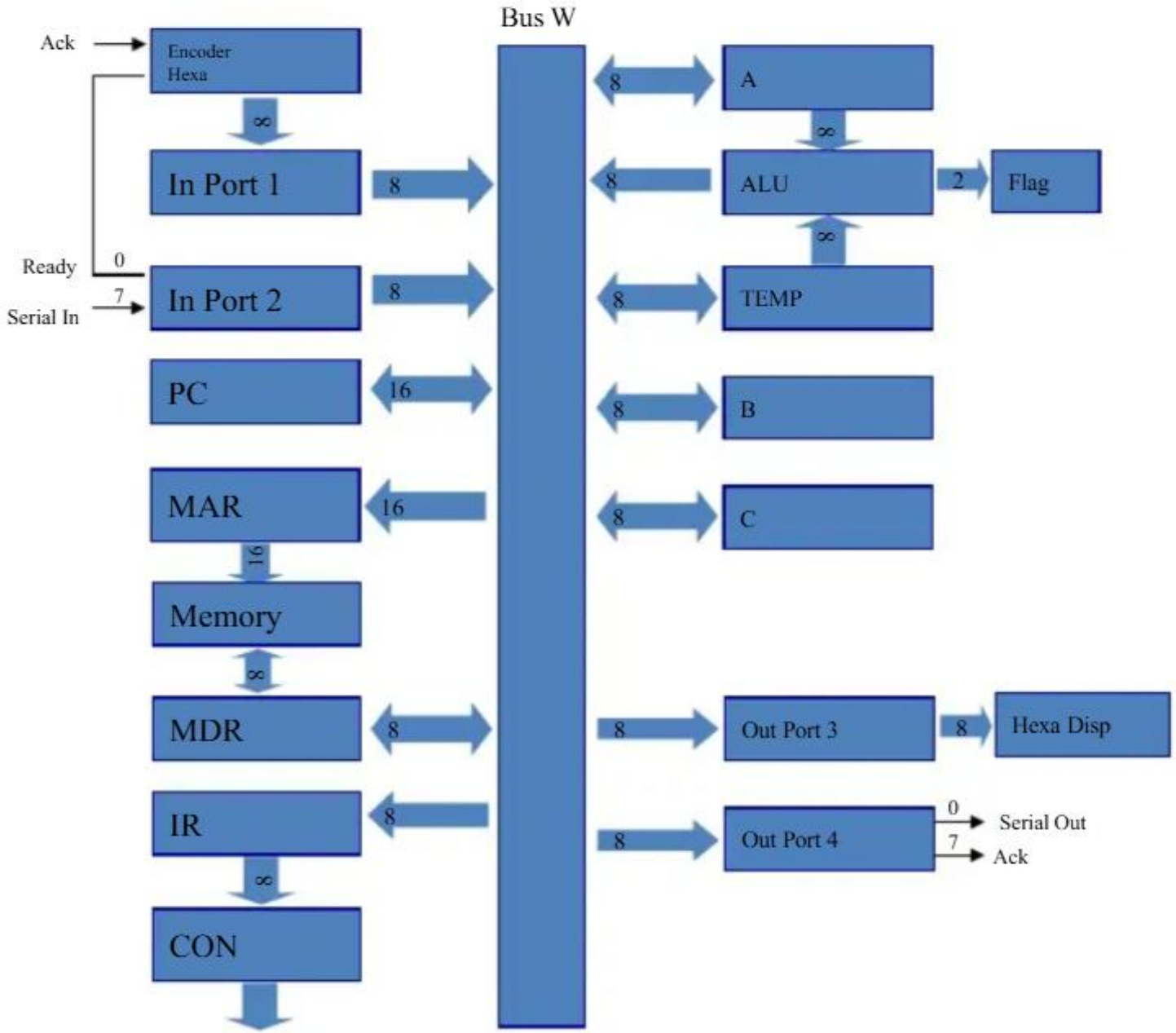
SAP 2 Architecture

- SAP 2 computers are designed by A. Malvino which is far more advanced than the SAP 1 computer.
- SAP 2 is an 8-bit computer with 64 KB (62 KB RAM and 2 KB ROM) memory and a 16-bit w-bus used as both address and data bus.
- SAP-2 has 2 input/output ports: port 1 and port 2.
- A hexadecimal keyboard encoder is connected to port 1.
- It allows us to enter hexadecimal instructions and data through port 1.
- The hexadecimal keyboard encoder sends a ready signal to bit 0 of port 2 (indicates the data in port 1 is valid).
- Port2: Serial In (Pin 7 of port 2)

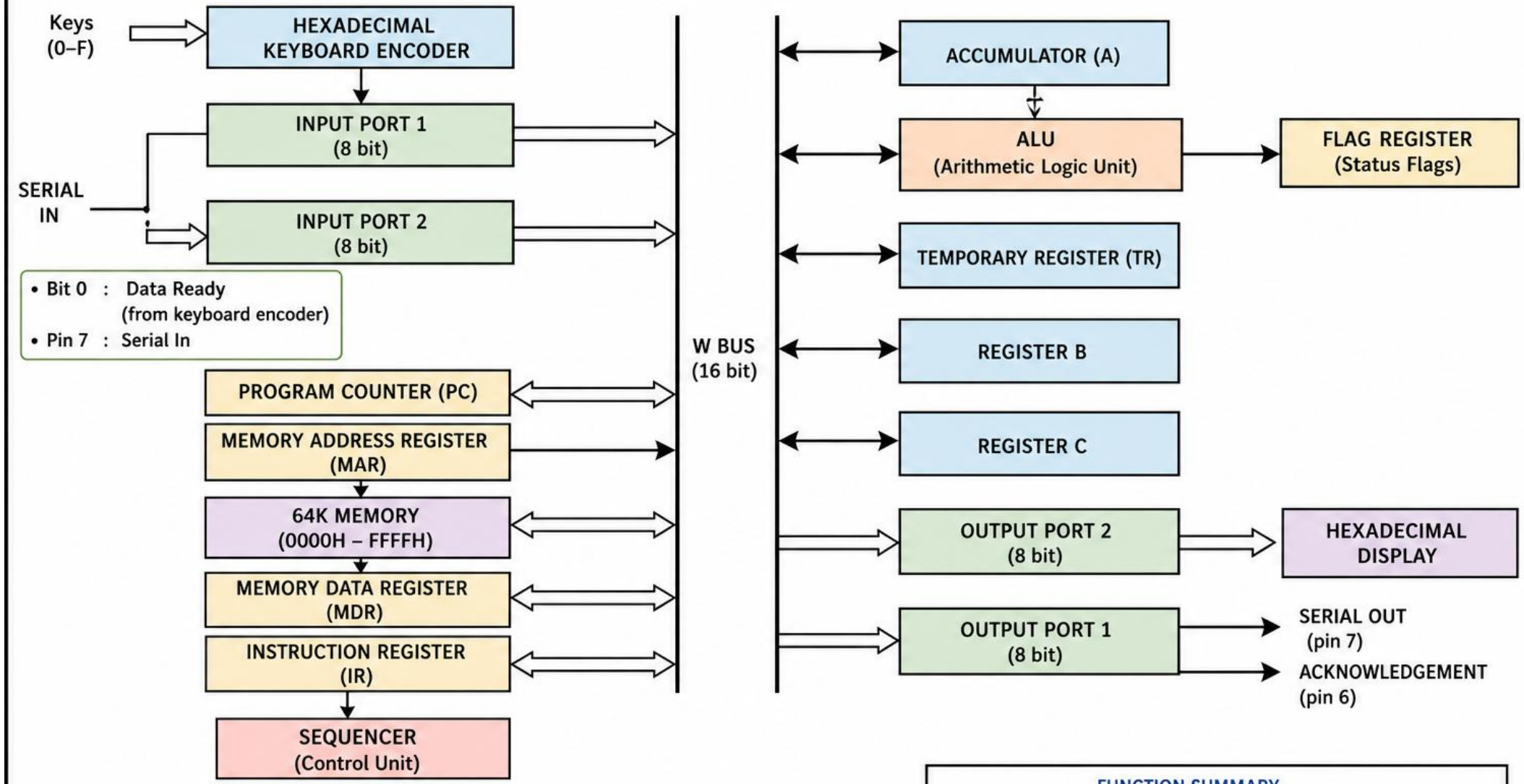
Features of SAP-2 Architecture

- Jump Instructions: loadable PC
- 16-bit program counter
- 8-bit op-code, 42 instructions
- 2 input ports- for hex keyboard in and serial in
- 2 output ports- for hex display and serial out
- 2 K ROM, up to 62K RAM (16-bit addresses) with read and write
- Memory data register (MDR) buffers reads and writes
- Accumulator can write to bus
- Temporary, B and C registers
- 16 arithmetic and logic operations in ALU
- Sign and zero flag

Architecture



SAP-2 COMPUTER ARCHITECTURE



LEGEND

Data flow (one way)

Data flow (both ways)

W BUS – Common 16-bit data bus connecting all registers, memory and I/O.

MAIN MEMORY

64K locations (16-bit each)

Address range : 0000H to FFFFH

(0 to 65535 decimal)

FUNCTION SUMMARY

- Data and instructions are entered via the hexadecimal keyboard.
- PC holds the address of the next instruction.
- MAR sends the address to memory; MDR transfers data to/from memory.
- IR holds the instruction; Sequencer controls the execution.
- ALU performs arithmetic/logic operations; Flags show the status.
- Results are sent to output ports for display or communication.

Components of SAP-2:

1. Input Ports:

- SAP-2 has 2 input/output ports: port 1 and port 2.
- A hexadecimal keyboard encoder is connected to port 1.
- It allows us to enter hexadecimal instructions and data through port 1.
- The hexadecimal keyboard encoder sends a ready signal to bit 0 of port 2.
- This signal indicates when the data in port 1 is valid.
- SERIAL input can be taken from pin 7 of port 2.

2. Program Counter (PC):

- Program Counter can store 16-bit address, therefore it can count from:
- PC = 0000 0000 0000 0000 0000 to PC = 1111 1111 1111 1111 1111
- This equivalent to 0000H to FFFFH (or decimal 0 to 65535)
- The low (CLR) signal resets the PC before each computer run, so the data processing starts with the instruction stored in memory location 0000H.

3. MAR and Memory:

- During the fetch cycle, the MAR receives 16 bit addresses from PC.
- The two state MAR output then address memory location.
- The memory has 2K ROM with address 0000H to 07FFH.
- The ROM contains a program called monitor that initializes the computer on power-up, interprets the keyboard inputs and so on.
- The rest of memory is 62 K RAM with addresses from 0800H to FFFFH.

4. Memory Data Register (MDR):

- The MDR is an 8-bit buffer register to store 8-bit Op-code.
- An 8-bit Op-code can accommodate 256 instructions.
- SAP-2 has only 42 instructions are identical with 8080/8085 instructions.
- Its output sets up the RAM.
- The MDR receives the data from the W-BUS before a write operation and it sends data to the BUS after a read operation.

5. Instruction Register (IR):

- Used for storing instructions read from the memory pointed by MAR.
- SAP-2 has more instructions (42) than SAP-1.
- Therefore, SAP-2 uses 8 bits for the Opcode rather than 4 bits.

6. Controller-Sequencer:

- The controller-sequencer produces the control word or micro-instructions that coordinates and direct the rest of the computer.
- Since, SAP-2 has bigger instruction set; the controller-sequencer has more hardware.
- The control word (CON) or micro-instructions determine how the registers react to the next positive edge clock.

7. Accumulator:

- The two-state output of the accumulator goes to the ALU and the three-state output to the W-bus.
- Therefore, the 8-bit word in the accumulator continuously drives the ALU, but this same word appears on the bus when E_A is active.

8. ALU and Flags:

- The ALUs are commercially available as integrated circuit (IC).
- These ALUs have 4 or more control bits that determine arithmetic and logic operation performed on word A and B.
- The ALU used in SAP-2 includes arithmetic and logic operation.
- Flag is a flip-flop that keeps track of a changing condition during computer run.
- The SAP-2 has two flags:
  - **Sign Flag:**
    - The sign flag is set when the accumulator contents become negative during the execution of some instructions.
  - **Zero Flag:**
    - The zero flag is set when the accumulator contents are zero.

9. TMP, B & C Register:

- Instead of using B register to hold the data being added to or subtracted from the accumulator a temporary (TMP) register is used.
- This allows us more freedom in using the B register.
- Besides these TMP and B registers SAP-2 includes a C register which gives us more flexibility in moving data during a computer run.

10. Output Ports:

- SAP-2 has two output ports: port 3 and port 4.
- The contents of the accumulator can be loaded into port 3, which drives the hexadecimal display.
- This allows us to see the processed data.
- The contents of the accumulator can also be sent to port 4.
- The pin 7 of port 4 sends ACKNOWLEDGE signal to the hexadecimal encoder.
- This ACKNOWLEDGE signal and READY signal are part of a concept called handshaking.
- The SERIAL OUT signal from pin 0 of port 4 converts parallel data in the accumulator into serial output data.

Difference Between SAP−1 and SAP−2 Architecture

Simple As Possible – 1 (SAP – 1)	Simple As Possible – 2 (SAP – 2)
It has 8-bit bus.	It has 16-bit bus.
Program Counter is 4-bit.	Program Counter is 16-bit.
It does not have hexadecimal keyboard encoder.	It has hexadecimal keyboard encoder.
It has single input.	It has two input port.
Memory Address Register receives 4-bit address from Program Counter.	Memory Address Register receives 16-bit address from Program Counter.
It does not have Read Only Memory (ROM).	It has 2 KB Read Only Memory (ROM).
It has 16 Byte memory.	It has 62 KB memory.
It does not have Memory Data Register (MDR).	It has Memory Data Register (MDR).
It has only adder/subtractor.	It has Arithmetic Logic Unit (ALU).
It does not have flag.	It has 2 flags.
It does not have temporary register.	It has temporary register.
It has single output port.	It has two output ports.
It has 5 instruction sets.	It has 42 instruction sets.

Bidirectional registers

- Bidirectional registers in the SAP-2 (Simple As Possible-2) microprocessor reduce bus capacitance and pin count by using a single set of wires for both input and output, connecting directly to the W-bus.
- They enable two-way data transfer, allowing registers to either load data from the bus or enable output to it.
- load and enable never simultaneously active
- on load, outputs are 3-state, input is taken from the bus
- on enable, inputs are ignored, output goes to the bus

Flags

- 2 flip flops, sign flag and zero flag
- set during arithmetic and logic operations to reflect final accumulator contents
- JM jumps only if the sign flag is set (minus result)
- JZ jumps only if the zero flag is set (zero result)
- JNZ jumps if the zero flag is clear (non-zero result)

Bidirectional Registers:

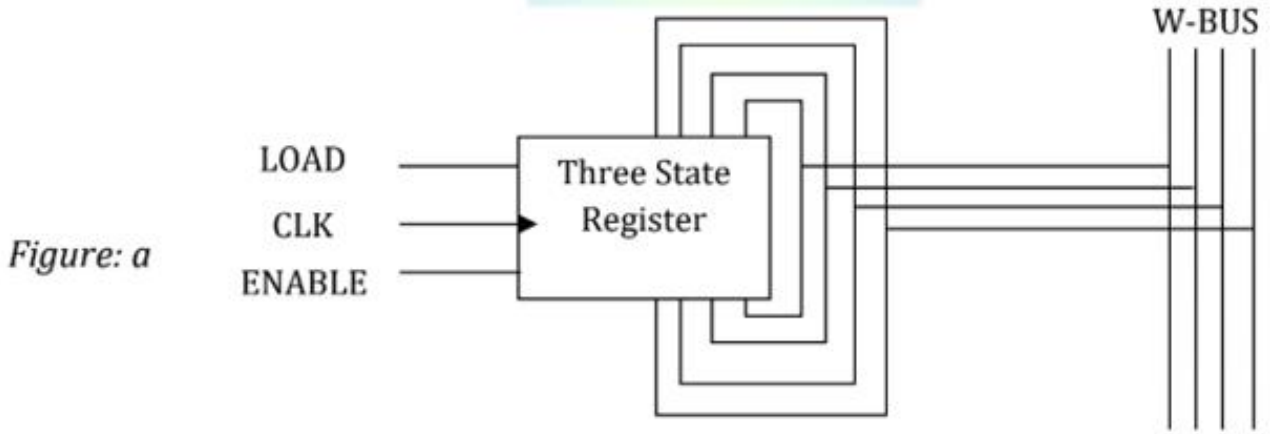


Figure: a

- To reduce the wiring capacitance of SAP-2, we will run one set of wires between each register and the bus. Figure (a) shows the idea.
- The input and output pins are shorted; only one group of wires are connected to the bus.

- The shorting of input and output pins does not affect the SAP-2 operation during computer run.
- During a computer run, either LOAD or ENABLE may be active, but not both at the same time.

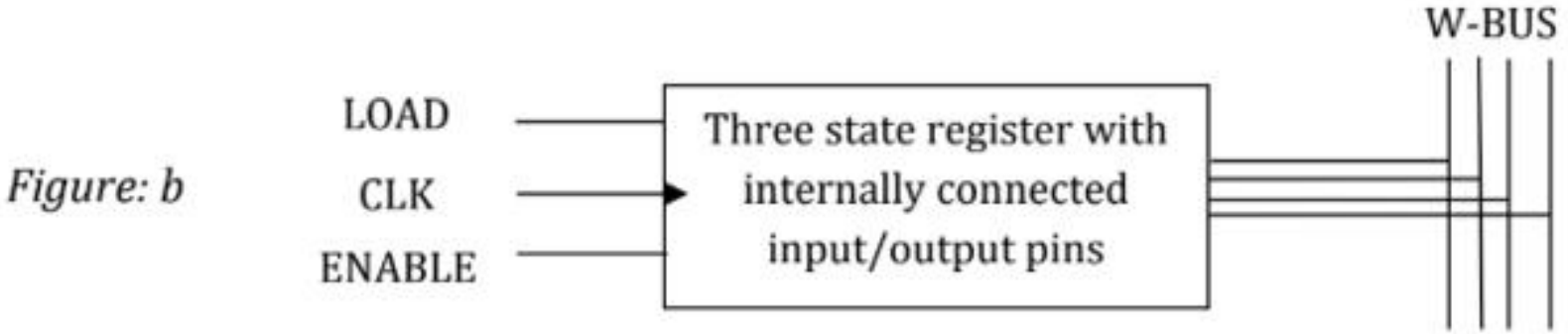


Figure: b

- An active LOAD means that a binary word flows from bus to the register input.
- During a load of operation, the output lines are floating.
- An active ENABLE means that a binary word flows from register to the bus.
- In this case the input lines floats.
- The IC manufacturer can internally connect the input and output pins of a three-state register.
- It reduces the wiring capacitance as well as the number of input/output pins.
- Figure (b) has four input/output pins instead of eight.

Figure: c

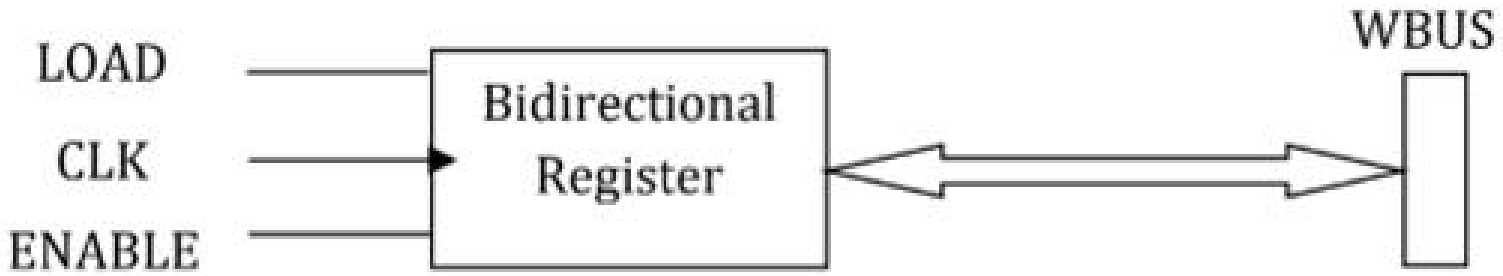


Figure: Bidirectional Register

- Figure (c) is the symbol of a three state register with internally connected input and output pins.
- The double headed arrow reminds us the path is bidirectional; the data can move either ways.

Flags of SAP 2 Architecture

- **Sign Flag (SF):** Set if the result of an operation is negative (most significant bit is 1), indicating a "minus" result.
- **Zero Flag (ZF):** Set if the result of the operation is zero.
- **Flag Management:** The flags are updated during ALU operations (like ADD, SUB) and also by INR (Increment) and DCR (Decrement) instructions.
- **Conditional Instructions:** Used by JM (Jump on Minus), JZ (Jump on Zero), and JNZ (Jump on Not Zero) to alter program flow.
- **Implementation:** The flag register is directly connected to registers A, B, and C to allow proper updating, particularly when flags are updated without going through the ALU, such as during the INR and DCR instructions.

End of Chapter One !